

M2. 바이트코딩 활용 앱/웹 개발

6. 바이트코딩 스킬 활용

Spec-Driven Development (SDD)

명확한 마크다운 스펙을 먼저 작성하고, AI코딩 에이전트가 그것을 구현하게 하는 방식

■ 3개 MD 파일 적용

구분	API
architecture.md	프로젝트의 전체 아키텍처와 기술적인 설계 기준을 정의하는 파일
prompts.md	AI가 프로젝트를 개발할 때 따라야 하는 작업 규칙과 프롬프트를 정의하는 파일
ui.md	UI/UX와 디자인에 대한 기준을 정의하는 파일

architecture.md

- 프로젝트 목적
- 핵심 기능
- 기술 스택
- 디렉터리 구조
- 데이터 모델
- API 설계
- 상태 관리 방식
- 인증 및 권한
- 비즈니스 로직
- 개발 제약사항

prompts.md

- AI 역할(Role)
- 개발 원칙
- 코드 스타일
- 구현 우선순위
- 리팩토링 기준
- 에러 처리 규칙
- 테스트 작성 기준
- 응답 형식
- 금지사항
- 자주 사용하는 프롬프트 템플릿

ui.md

- 디자인 컨셉
- 컬러 시스템
- 타이포그래피
- 컴포넌트 규칙
- 레이아웃
- 반응형 기준
- 페이지별 UI 요구사항
- 인터랙션
- 접근성 기준

SDD 방법론 적용 효과



AI 기억력 보완

- MD = AI의 장기 기억 장치. @참조 한 번으로 모든 맥락을 AI에게 전달. 설명 반복 끝!



일관성 유지

- 스타일·구조·규칙이 흔들리지 않음. 10번째 기능도 1번째와 같은 스타일로 생성됨



시간 절약

- 매번 설명 반복 없이 @참조 한 번으로 끝. 설명 시간 대신 기능 추가에 집중 가능



협업 가능

- 팀원이 MD 3개만 읽으면 프로젝트 파악 완료. 온보딩 시간 대폭 단축



내 자산이 됨

- MD는 코드보다 오래 쓰는 설계 문서. 다음 프로젝트에도 템플릿으로 재사용 가능

MD 파일 3개 작성에 드는 1시간이 이후 수십 시간을 절약!

architecture.md 예시

```
# architecture.md - 기술 설계도

## 기술 스택
- - Backend : Python 3.x + Flask
- Frontend : 순수 HTML + CSS + JavaScript
- 통신 방식: JavaScript fetch API → Flask JSON 응답
- Database : SQLite (todos.db 파일, 자동 생성)

## 폴더 구조
/todo-app
  app.py           → Flask 앱 메인 파일 (라우트 + DB 처리)
  todos.db         → SQLite 데이터베이스 파일 (자동 생성)
  /static
    index.html     → 단일 페이지 (HTML)
    style.css      → 전체 스타일
    main.js        → 할일 추가·삭제·체크·필터 JavaScript

## DB 스키마 - todos 테이블
id(uuid PK) | title(text) | category(text) | due_date(date)
is_done(boolean DEFAULT false) | created_at(timestamp)

## API 설계
GET /api/todos      → 전체 조회      POST /api/todos      → 추가
PATCH /api/todos/[id] → 완료 토글    DELETE /api/todos/[id] → 삭제
```

prompts.md 예시

```
# prompts.md - AI 지시 규칙 & 프롬프트 레시피

## AI가 항상 지킬 규칙
- Python 코드는 함수 단위로 깔끔하게 분리
- Flask API는 항상 JSON으로 응답 { "data": ..., "error": null }
- 에러는 try-except로 처리, 에러 메시지는 한국어로
- HTML은 /static/index.html 한 파일
- CSS는 /static/style.css 한 파일
## 프롬프트 템플릿

### 새 기능 추가 (Plan 모드 먼저)
@ui.md @architecture.md @prompts.md 를 참고해서
[기능명] 기능을 추가하는 계획을 만들어줘.
- 수정할 파일 목록
- 각 파일에서 변경할 내용 요약
계획이 맞으면 코드를 작성해줄게.
```

ui.md 예시

```
# ui.md - 화면 설계도

## 색상 & 폰트
- 주색: #2563EB (파랑)   배경: #FFFFFF   텍스트: #1A1A1A
- 완료 항목: #94A3B8 (회색)   경고(마감 지남): #DC2626 (빨강)
- 폰트: Pretendard / 제목 20px Bold / 본문 14px Regular

## 메인 페이지 (/)
- 헤더: 로고 "오늘의 할일" (좌) + "+ 추가" 버튼 (우, 파란색)
- 카테고리 탭: [전체] [업무] [개인] [공부]
- 빈 상태: "할일이 없어요! 추가해보세요 😊"

## 할일 카드 컴포넌트
- 체크박스(좌) + 제목 + 카테고리 배지 + 마감일(우)
- 완료 시: 취소선 + 전체 회색   마감 지남: 마감일 텍스트 빨간색

## 컴포넌트 규칙
- 버튼: rounded-lg px-4 py-2   카드: shadow-sm border rounded-xl p-4
```

3개 MD 파일 완성! → Chat에서 @참조하며 기능 만들기 시작!

마크다운(MarkDown)

- 특수기호로 문서 구조를 표현하는 가벼운 문법
- 사용문법
 - # 하나면 큰 제목 됨
 - **별표 두 개**면 굵게 됨
 - -(하이픈)이면 목록 됨
 - 문법이 단순해서 실험 기록에 잘 맞음

"글을 꾸미는 것"보다 "구조를 만드는 것"에 가까움

참고. 마크다운 예시

기능	입력	결과
제목(H1)	# 제목	제목
제목(H2)	## 제목	제목
제목(H3)	### 제목	제목
굵게	**굵은 글씨**	굵은 글씨
기울임	*기울임*	<i>기울임</i>
취소선	~~삭제~~	삭제
굵게 + 기울임	***강조***	<i>강조</i>
인라인 코드	`print()`	print()
코드 블록	python print("Hello") 	코드 블록 생성
순서 없는 목록	- 사과	사과
순서 있는 목록	1. 사과	1. 사과

기능	입력	결과
체크박스	- [x] 완료	☒ 완료
체크박스	- [] 미완료	☐ 미완료
링크	[OpenAI](https://openai.com)	OpenAI
이미지	![설명](image.png)	이미지 표시
인용문	> 인용문	> 인용문
수평선	---	_____
표	이름 나이	표 생성
이스케이프	*별표*	*별표*

md 파일 작성(프롬프트)

■ STEP 1 — Cursor Chat (Ctrl+L) 열고 아래 프롬프트 입력



나는 "오늘의 할일" 앱을 만들 거야.

기능: 할일 추가, 완료 체크, 삭제, 카테고리(업무/개인/공부) 분류, 마감일 설정

- Backend: Python Flask

- Frontend: 순수 HTML + CSS + JavaScript

- Database: SQLite3

아래 3개 MD 파일 초안을 각각 작성해줘.

파일에 바로 붙여넣을 수 있게 마크다운 형식으로, 구체적으로 작성해줘.

1. ui.md: 페이지 목록, 컴포넌트 구조, 색상·폰트 규칙

2. architecture.md: 기술 스택, 폴더 구조, API 엔드포인트, DB 스키마

3. prompts.md: AI 코딩 규칙, 자주 쓸 프롬프트 템플릿

■ STEP 2 — 생성된 3개 파일 확인

📁 프로젝트이름

/todo-app/

├── ui.md

├── architecture.md

└── prompts.md

Skill이란?

■ AI 코딩 도구(Cursor, Codex 등)가 특정 작업을 수행할 때 참조하는 구조화된 지식 문서(작업 매뉴얼)

1

매번 고민할 필요 없음

"어떻게 물어보지? 어떤 프롬프트를 쓰지?" → 스킬이 그 역할을 대신 수행

2

비개발자도 전문가 수준

질문·문서화를 비개발자도 바로 전문가 수준으로 제작 가능

3

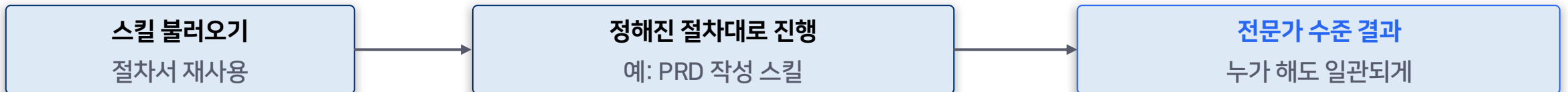
검증된 절차 재사용

좋은 절차는 이미 스킬로 만들어져 있음 → 잘 만든 것을 활용하는 것도 실력

Before (스킬 없이)



After (스킬 활용)



Skill 실제 파일 구조 예시

■ Markdown 파일(.md) 형태

```
# grill.me Skill
```

```
## 역할
```

당신은 아이디어 검증 전문가임.

```
## 규칙
```

1. 아이디어 입력받으면 7가지 질문 순서대로 함.
2. 질문은 한 번에 하나씩만 함.

```
## 질문 목록
```

1. 누가 씀? 2. 어떤 문제를 해결함?

```
## 출력 형식
```

****문제 정의****: {답변 기반으로 정리}

****대상 사용자****: {답변 기반으로 정리}

```
---
name: to-prd
description: Turn the current conversation into a PRD and publish
disable-model-invocation: true
---

This skill takes the current conversation context and codebase un
The issue tracker and triage label vocabulary should have been pro

## Process

1. Explore the repo to understand the current state of the codebas
2. Sketch out the seams at which you're going to test the feature.
Check with the user that these seams match their expectations.
3. Write the PRD using the template below, then publish it to the

<prd-template>

## Problem Statement

The problem that the user is facing, from the user's perspective.

## Solution

The solution to the problem, from the user's perspective.

## User Stories

A LONG, numbered list of user stories. Each user story should be

1. As an <actor>, I want a <feature>, so that <benefit>
```

참고. 주요 Skill

기능	특징	주요기능	웹주소
grill-me	코드 작성 전 설계를 집요하게 검증하는 사전 검토 도구	① 계획/설계를 질문으로 집요하게 검증 ② 결정 트리 분기별로 하나씩 해소 ③ 코드베이스 탐색으로 중복 질문 자동 생략	skills.sh/mattpocock/skills/grill-me
to-prd	추가 질문 없이 대화만으로 PRD 완성 후 이슈 자동 등록	① 대화 맥락만으로 PRD 자동 생성 ② 사용자 스토리·구현 결정·테스트 계획 포함 ③ 이슈 트래커에 needs-triage 라벨로 자동 게시	skills.sh/mattpocock/skills/to-prd
diagnose	6단계 체계로 어려운 버그를 끝까지 추적하는 디버깅 규율	① 피드백 루프 구축이 핵심 — 재현 가능한 신호 확보에 최우선 ② 3~5개 가설을 먼저 나열하고 순위 매긴 뒤 검증 ③ 수정 전 회귀 테스트 작성 → 수정 → 디버그 코드 전부 제거	skills.sh/mattpocock/skills/diagnose
frontend-design	Anthropic 공식, 디자인 결정을 AI가 자동 판단	① UI 색상/타이포그래피/레이아웃 가이드 적용 ② 디자인 일관성 자동 유지 ③ 컴포넌트 스타일 결정 지원	skills.sh/anthropics/skills/frontend-design
vercel-react-best-practices	Vercel 공식, 배포 환경까지 고려한 React 패턴	① Vercel 최적화 React 패턴 ② 성능/렌더링 모범 사례 ③ 배포 친화적 코드 구조	skills.sh/vercel-labs/agent-skills/vercel-react-best-practices
web-design-guidelines	접근성·반응형·UX를 종합한 웹 디자인 원칙	① 웹 접근성 및 반응형 원칙 ② UX 중심 설계 가이드 ③ 크로스브라우저 호환성 고려	skills.sh/vercel-labs/agent-skills/web-design-guidelines
systematic-debugging	감이 아닌 체계적 가설 검증으로 버그 추적	① 버그 원인을 단계별로 좁혀가는 프로세스 ② 가설 설정 → 검증 → 범위 축소 반복 ③ 디버깅 과정 문서화 지원	skills.sh/obra/superpowers/systematic-debugging

참고. 주요 Skill

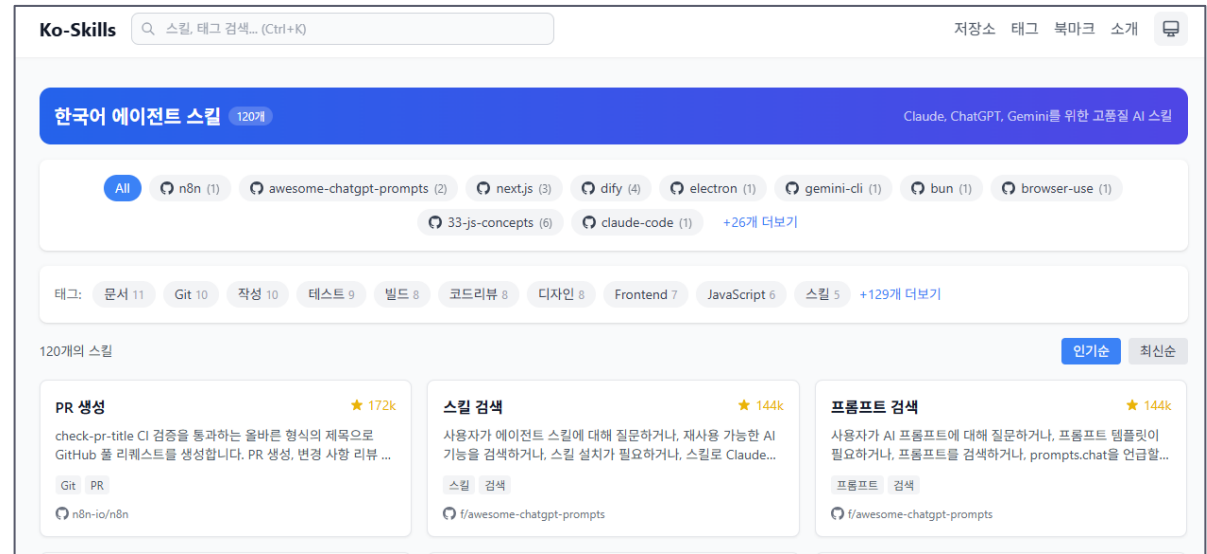
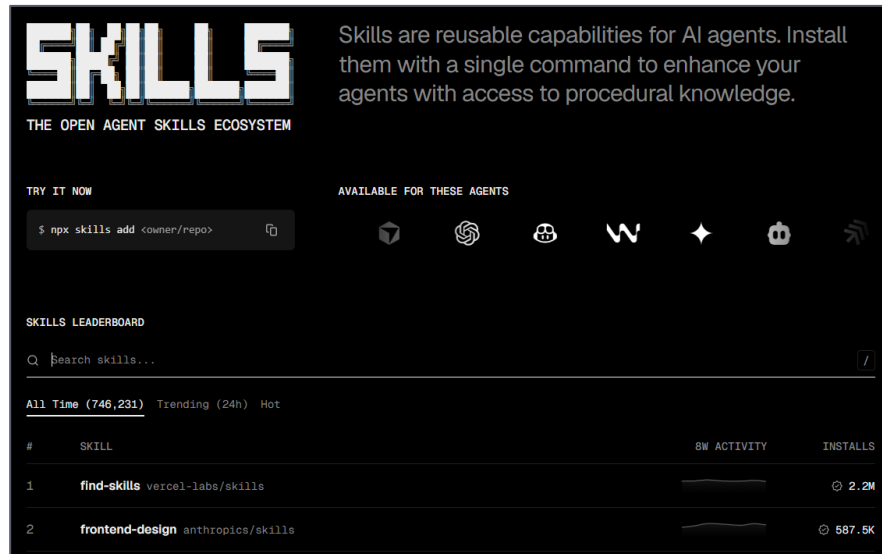
기능	특징	주요기능	웹주소
shadcn	shadcn 공식, 컴포넌트 설치부터 커스터마이징까지	① shadcn/ui 컴포넌트 올바른 설치/사용법 ② 테마 커스터마이징 가이드 ③ Radix UI 기반 접근성 패턴	skills.sh/shadcn/ui/shadcn
tdd	테스트를 먼저 쓰는 Red-Green-Refactor 루프 강제	① Red-Green-Refactor 루프 적용 ② 테스트 먼저 작성 후 구 ③ 통합·단위 테스트 전략	skills.sh/mattpocock/skills/tdd
pdf	Anthropic 공식, PDF 생성부터 추출까지 올인원	① PDF 생성/병합/분할 ② 텍스트·이미지 추출 ③ 폼 필드 작성 및 암호화	skills.sh/anthropics/skills/pdf
pptx	Anthropic 공식, PPTX 생성·편집·파싱 전반 지원	① 슬라이드 덱 생성 및 편집 ② 레이아웃·노트·템플릿 처리 ③ 기존 PPTX 파싱 및 내용 추출	skills.sh/anthropics/skills/pptx
seo-audit	SEO 구조를 자동 분석해 우선순위별 개선안 제시	① 페이지 SEO 구조 분석 ② 개선 우선순위 제안 ③ 메타/구조화 데이터 점검	skills.sh/coreyhaines31/marketingskills/seo-audit
docx	Anthropic 공식, Word 문서 생성·편집 전담	① Word 문서 생성 및 서식 적용 ② 목차·페이지번호·헤더 처리 ③ 기존 .docx 편집 및 내용 추출	skills.sh/anthropics/skills/docx

Skill 활용 방법

■ 활용 순서

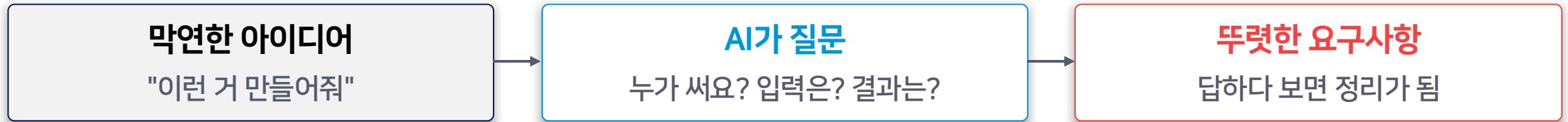


■ Skill 검색 사이트 : <https://www.skills.sh>, <https://ko-skills.pages.dev>



grill-me 스킬을 활용한 아이디어 구체화

- 구체화가 필요한 이유 : 막연한 아이디어 한 줄로 요청시, 빈칸은 AI가 제멋대로 채움
- grill-me 스킬을 활용한 아이디어 구체화
 - 내가 AI에게 요청하는 게 아니라 **AI가 나에게 필요한 기술을 물어봄**



- 스킬 설치 링크: <https://www.skills.sh/mattpocock/skills/grill-me>

1. Cursor Chat에 grill-me 스킬을 활용해서 아이디어만 입력하면 됨

 /grill-me 냉장고 재료 입력하면 만들 수 있는 요리 추천하는 앱 만들고 싶어

2. grill-me가 던지는 질문들

- 누가 씬?
- 어떤 문제가 있음?
- 기존 서비스랑 뭐가 다름?
- 젤 중요한 기능 하나만 꼽으면?
- 첫 번째 버전(MVP)은 어디까지임?
- 성공 기준이 뭐임?

3. 대답하다 보면 아이디어가 구체화되고, 최종적으로 아이디어 검증 문서가 나옴

- 문제 정의 / 대상 사용자
- 사용자 니즈 / 핵심 가치
- MVP 범위 / 제외 기능
- 성공 기준 / 위험 요소

to-spec(PRD) 스킬을 활용한 요구사항 생성

- PRD(Product Requirements Document)
- "무엇을 만들 것인가"를 적은 요구사항 목록 문서임 → AI가 판단할 근거가 되는 문서

항목	개념	예시(쇼핑앱)
왜	서비스를 만드는 이유와 목표를 정의함	모바일 중심 고객이 빠르고 간편하게 상품을 구매할 수 있는 환경 제공이 목적임
문제	현재 사용자가 겪는 불편이나 해결해야 할 문제를 정의함	오프라인·SNS 주문만 가능함 주문·재고를 수기로 관리하여 실수 잦음
사용자	서비스를 사용하는 대상과 역할을 정의함	일반 고객(상품 구매), 재구매 고객(빠른 재주문), 관리자(상품·주문·재고 관리)
범위	개발에 포함할 기능과 제외할 기능을 정의함	포함: 회원가입, 상품 조회, 장바구니, 결제, 관리자 화면제외: 리뷰, 채팅, 다국어 지원
동작 방식	서비스가 어떤 규칙과 흐름으로 동작하는지 정의함	결제 실패 시 장바구니 유지함 재고 부족 시 결제 전에 안내함 주문 완료 시 재고 자동 차감함
성공 기준	기능이 성공했는지 판단하는 기준을 정의함	결제까지 클릭 3회 이내 완료 목표 재구매율 20% 이상 달성 목표

to-spec(PRD) 스킬을 활용한 요구사항 생성

- PRD만 가지고 바로 코딩할 때 흔히 빠지는 부분 → 보완

누락 항목	발생되는 문제
데이터 모델/스키마	필드, 타입, 관계가 불명확해 AI가 임의로 설계함
에러/예외 처리 기준	실패 메시지, 재시도 방식 등을 AI가 임의로 결정
인증/권한 경계	누가 뭘 보고 수정할 수 있는지 불명확
비기능 요구사항	성능, 동시성, 데이터 규모(100명 vs 100만명)에 따라 설계가 완전히 달라짐
기존 시스템 연동 방식	기존 API·DB·인증 시스템과의 연결 방식 누락

to-spec(PRD) 스킬을 활용한 요구사항 생성

- to-spec: 아이디어 검증 결과를 바탕으로 제품 요구사항 문서(PRD)를 생성해 주는 스킬
- grill-me 스킬에서 나온 결과물을 바탕으로 개발에 필요한 기준들이 정리되어 파일로 나옴



지금까지 내용을 SPEC.md파일에 작성해줘

- 스킬 설치 링크: <https://skills.sh/mattpocock/skills/to-spec>

skills / mattpocock / skills / to-spec

to-spec

INSTALLATION Command Prompt

```
$ npx skills add https://github.com/mattpocock/skills --skill to-spec
```

SKILL .md

This skill takes the current conversation context and codebase understanding and produces a spec (you may know this document as a PRD). Do NOT interview the user — just synthesize what you already know.

The issue tracker and triage label vocabulary should have been provided to you — run `/setup-matt-pocock-skills` if not.

Process

1. Explore the repo to understand the current state of the codebase, if you haven't already. Use the project's domain glossary vocabulary throughout the spec, and respect any ADRs in the area you're touching.
2. Sketch out the seams at which you're going to test the feature. Existing seams should be preferred to new ones. Use the highest seam possible. If new seams are needed, propose them at the highest point you can. The fewer seams across the codebase, the better - the ideal number is one.

Check with the user that these seams match their expectations.

3. Write the spec using the template below, then publish it to the project issue tracker. Apply the `ready-for-agent` triage label - no need for additional triage

실습 예제. 냉장고 재료로 요리 추천하는 앱

아이디어: 냉장고에 남은 재료를 입력하면 만들 수 있는 요리를 추천해주는 앱을 만들고 싶다.

■ 커서에 입력할 프롬프트

NO.	프롬프트
1	/grill-me 냉장고의 재료로 만들 수 있는 요리 추천 앱 만들고 싶어
2	1번부터 질문을 체크할 수 있게 물어봐 줘
3	/to-spec 지금까지 내용을 SPEC.md파일에 작성해줘
4	SPEC.md 내용 검토 후 필요한 내용 추가
5	@SPEC.md 내용으로 코드를 작성해

#참고. Plan 모드 사용 및 검토 사항

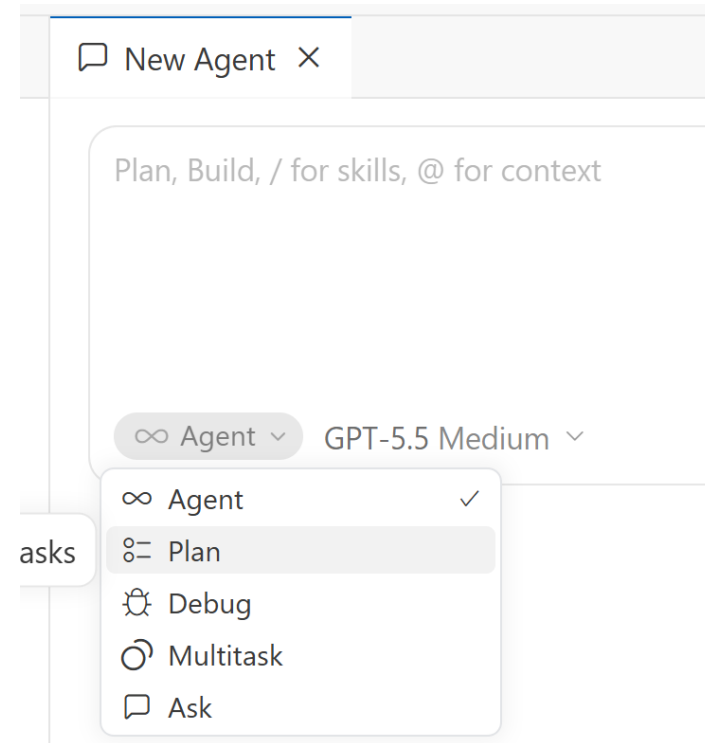
- AI가 코드를 바로 짜지 않고, "이렇게 만들겠습니다"라는 계획을 먼저 보여주는 모드

- plan모드 사용법

1. Composer 열기 (Ctrl+I)
2. 상단 드롭다운에서 "Plan" 선택
3. @MD 파일 참조 + 프롬프트 입력
4. AI가 계획을 텍스트로 보여줌 (코드는 아직 없음)
5. 계획 검토 → 문제 있으면 수정 요청 → 문제 없으면 "진행해줘" 또는 Agent 모드로 전환
6. 실제 코드 생성

- Plan 모드에서 확인할 사항

- 만들려는 파일 목록이 맞는가?
- 수정하려는 파일이 올바른가? (엉뚱한 파일 건드리면 안 됨)
- 접근 방식이 architecture.md와 일치하는가?
- 빠진 기능이나 조건은 없는가?



#참고. Plan 모드 예시

■ Plan 모드 프롬프트

```
@ui.md @architecture.md @prompts.md
```

할일 추가 기능을 만들 계획을 세워줘.

- 어떤 파일을 만들고 수정하는지
- 각 파일에서 무엇을 작성하는지
- Flask API와 JavaScript가 어떻게 연결되는지 계획만 먼저 보여줘. 코드는 아직 작성하지 마.

■ Plan 검토 체크리스트

파일 관련

- ☐ 만들거나 수정하는 파일 목록이 architecture.md와 일치하는가?
- ☐ 건드리면 안 되는 파일을 건드리지 않는가?
- ☐ 새로 만드는 파일이 올바른 폴더에 위치하는가?

기능 관련

- ☐ 내가 요청한 기능이 계획에 모두 포함됐는가?
- ☐ 빠진 조건이나 엣지 케이스는 없는가?
- ☐ API URL이 architecture.md의 설계와 맞는가?

흐름 관련

- ☐ JavaScript → Flask API → SQLite 흐름이 맞는가?
- ☐ 성공/실패 처리가 계획에 포함됐는가?
- ☐ 화면 업데이트(DOM) 방식이 자연스러운가?

■ 체크리스트 검토 후

- 이상 있으면: 3번 항목에서 API URL을 /api/todos로 바꿔줘. 그리고 삭제 확인 창(confirm)이 빠져있는데 추가해줘.
- 이상 없으면: 계획대로 진행해줘

M3. 바이트코딩 활용 RAG 구현

7. RAG 감 잡기

AI는 왜 우리 회사 일을 모를까

▶ LLM이 배운 것과 우리가 묻는 것 사이의 간극



LLM이 배운 것

- 인터넷에 공개된 문서 · 책 · 코드
- 학습을 끝낸 시점까지의 정보
- 누구에게나 똑같이 주어지는 일반 지식

→ 세상의 평균적인 지식



우리가 묻는 것

- 우리 회사 연차·출장 규정
- 우리 팀이 지난달 쓴 보고서
- 어제 회의록, 이번 분기 제품 사양서

→ 인터넷에 없는, 우리만의 지식

요즘 AI는 이 경계를 스스로 압니다. 모르면 "모른다"고 말합니다.
그리고 "무엇을 봐야 하는지"까지 알려줍니다.

✅ 답할 수 있는 것

"근로기준법상 연차는 15일입니다" 공개된 법·기준·일반 상식 → 학습 데이터에 들어 있음

❌ 답할 수 없는 것

"재택근무 신청 기한은 회사마다 다릅니다. 확인할 수 없습니다" → 우리 문서에만 있는 정보

🔑 AI가 덧붙이는 말

"사내 복리후생·근무제도 안내를 확인하셔야 합니다" → 필요한 게 뭔지는 AI도 압니다

【실습 ①】 AI가 답할 수 있는 것과 없는 것

▶ 전부 휴가·근무 질문인데 반응이 갑니다

lab1_no_doc 폴더만 열고 새 채팅에서 시작 — 상위 폴더를 열면 Cursor가 옆 폴더의 문서를 찾아냅니다

A조 · 공개된 지식

A-1 "근로기준법상 1년 이상 근무한 직원의
연차유급휴가는 며칠인가요?"

A-2 "주 52시간제에서 1주 연장근로 한도는?"
→ 답합니다. 법과 공개 기준은 학습되어 있습니다

B조 · 우리 문서에만 있는 지식

B-1 "재택근무는 며칠 전까지 신청하고
한 달에 최대 며칠까지?"

B-2 "하계휴가 지원금은 얼마인가요?"
→ 못 답합니다. "확인할 수 없습니다"

#	AI가 답을 줬나?	답을 줬다면 근거는? / 못 줬다면 이유는?
A-1 / A-2	답함 / 못 함	
B-1 / B-2	답함 / 못 함	

★ 마지막 단계 — B조에서 AI가 "무엇을 보라"고 했나요? 그 문장을 그대로 적으세요

"정확한 신청 기한과 월 한도는 사내 복리후생·근무제도 안내나 인사 담당 확인이 필요합니다"

해답은 「오픈북 시험」

▶ 모델을 다시 가르치는 대신, 답할 때 참고자료를 쥐여준다

❑ 폐쇄형 시험 = 그냥 LLM

- 머릿속에 든 것만으로 답한다
- 모르는 것도 일단 쓴다 (빈칸은 0점이니까)
- 우리 회사 규정은 애초에 배운 적이 없다

→ 그럴듯한 오답

❑ 오픈북 시험 = RAG

- 문제를 보고 필요한 페이지를 먼저 찾는다
- 찾은 페이지를 보면서 답을 쓴다
- 책에 없으면 "없다"고 쓸 수 있다

→ 근거 있는 답

RAG (Retrieval-Augmented Generation) · 검색 증강 생성

AI를 다시 학습시키는 것이 아니라, 질문이 들어온 순간 우리 문서에서 관련 부분을 찾아 질문과 함께 건네주는 방식

※ 재학습(파인튜닝)과의 차이 — 문서가 바뀌면 다시 학습시킬 필요 없이 창고의 파일만 갈아 끼우면 됩니다.

Retrieval · 검색

우리 문서에서 관련 부분을 찾고



Augmented · 증강

질문에 그 내용을 덧붙여서



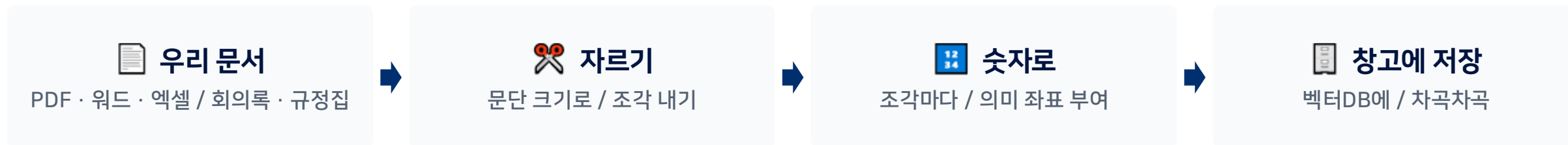
Generation · 생성

LLM이 그것을 보고 답을 만든다

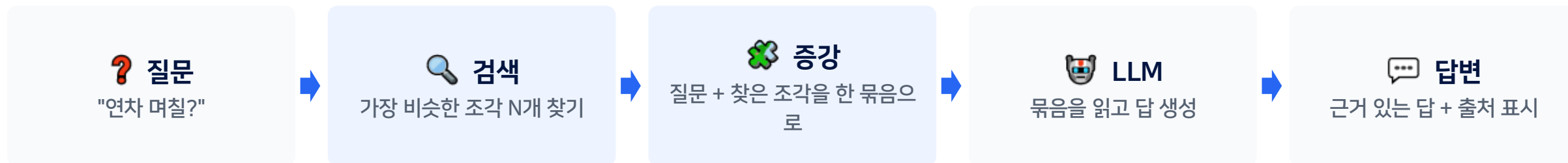
RAG는 이렇게 동작한다

▶ 사전 준비(참고 만들기)와 질문 시점(꺼내 쓰기)을 나눠서 본다

① 사전 준비 — 한 번만 해두면 되는 일



② 질문이 들어온 순간 — 매번 반복되는 일



기억할 한 문장

RAG는 AI를 똑똑하게 만드는 기술이 아니라, AI가 답하기 직전에 「정답이 적힌 페이지」를 책상 위에 올려주는 기술입니다.

그래서 문서가 최신이면 답도 최신이고, 문서에 없으면 "없다"고 답하게 만들 수 있습니다.

【실습 ②】 AI가 보라던 그 문서를 준다

▶ 실습 ①의 B조 질문을 그대로 다시

STEP 1. 폴더 전환

lab1_no_doc 닫고 lab2_with_doc
열기 한빛소재 규정 PDF (9쪽)

STEP 2. 같은 질문 다시

PDF를 채팅창에 끌어다 놓고 실습 ①
의 B-1, B-2를 토씨 하나 바꾸지 말고
그대로

STEP 3. 답변 비교

문서 없을 때 vs 있을 때 무엇이 달라
졌는지 아래 표에 기록

STEP 4. 근거 요구

"몇 조 몇 항인지 알려줘"를 덧붙이고
PDF에서 직접 찾아 맞는지 확인

비교 항목	문서를 주지 않았을 때	문서를 주었을 때
B-1 재택근무	"확인할 수 없습니다"	
B-2 하계휴가 지원금	"확인할 수 없습니다"	
출처(조항 번호)	없음	

✅ 여기서 느껴야 할 것

바뀐 것은 AI가 아닙니다. 같은 AI, 같은 질문. AI 손에 문서가 있느냐 없느냐
하나뿐입니다.

★ 마지막에 꼭 물어볼 질문

"사내 어린이집 운영 시간은?" — 문서에 없는 내용입니다. "없다"고 할까요,
지어낼까요?

왜 문서를 통째로 넣으면 안 될까

▶ AI의 책상은 생각보다 좁습니다 — 컨텍스트 윈도우

AI가 한 번에 볼 수 있는 양은 정해져 있다

- 모델마다 "한 번에 읽을 수 있는 글자 수" 상한이 있음
- 이 상한에는 질문 + 첨부 문서 + 이전 대화가 모두 포함
- 넘치면 앞부분이 잘려 나가거나 오류가 납니다
- 참고: 널리 쓰이는 모델 기준 대략 A4 100~수백 장 수준

통째로 넣으면 생기는 3가지 문제

- ① 잘림 — 정작 중요한 뒷부분이 사라진다
 - ② 비용 — 넣는 글자 수만큼 요금이 붙는다
 - ③ 산만 — 관계없는 내용까지 읽느라 답이 흐려진다
- ※ 바이트코딩 중 "AI가 앞에서 정한 규칙을 까먹는" 현상도 같은 이유

그래서 "문서 전부"가 아니라 "질문과 관련된 부분만" 골라 넣습니다.
골라 넣으려면 ① 잘라두고 ② 찾을 수 있게 만들어 줘야 합니다.

① 자른다 — 청크(Chunk)

두꺼운 매뉴얼을 문단 단위로 잘라 「조각 카드」를 만드는 일

- 너무 크게 자르면 → 필요 없는 내용까지 딸려 옴
- 너무 잘게 자르면 → 문장이 끊겨 뜻을 잃음
- 조각끼리 앞뒤를 조금 겹쳐 둔다(overlap) → 문맥 유지

② 숫자로 바꾼다 — 임베딩(Embedding)

조각마다 「의미의 좌표」를 매겨 컴퓨터가 비교할 수 있게 만드는 일

- 뜻이 비슷한 문장은 좌표도 가깝게 찍힌다
- 그래서 단어가 달라도 뜻이 같으면 찾아낼 수 있다
- 이 좌표들을 담아두는 창고가 벡터DB(Vector Store)

키워드 검색과 무엇이 다른가

▶ 단어가 달라도 뜻이 같으면 찾아내는 것 — 의미 검색

	기존 키워드 검색 (Ctrl+F)	의미 검색 (임베딩 기반)
찾는 방식	글자가 똑같이 들어있는 곳을 찾음	뜻이 가까운 곳을 찾음
"연차" 검색 시	"연차"가 적힌 문장만 나옴	"휴가 일수", "유급 휴가 부여" 문장도 함께 나옴
오타·유사어	못 찾음 ("년차"로 치면 0건)	대체로 찾아냄
한계	단어를 정확히 알아야 함	엉뚱하게 가까운 문장이 섞이기도 함

🔑 비유 — 도서관에서 책 찾기

키워드 검색은 「제목에 그 단어가 든 책」을 찾아주는 사서이고, 의미 검색은 「그 주제를 다루는 책」을 찾아주는 사서입니다.

RAG의 검색 단계는 후자의 사서를 쓰기 때문에, 질문에 쓴 단어가 문서에 없어도 답을 찾아냅니다.

🔑 10번 시간에 이 단어들이 이렇게 쓰입니다

"PDF를 500자 단위로 잘라(청크) 임베딩한 다음 벡터DB에 저장하고, 질문과 가장 비슷한 조각 5개를 찾아(검색) 답하게 만들어줘"

— 이 한 문장을 Cursor에게 말할 수 있으면, 오늘 7번의 목표는 달성입니다.

【실습 ③】의미 검색 체험하기

▶ 한빛소재 규정집의 실제 문장으로 · lab3_semantic_search 폴더

라운드 1 · "재택근무는 며칠 전에 신청해야 하나요?"

- ① 해외 출장자는 출발 14일 전까지 출장계획서를 제출
 - ② 임직원은 월 8일의 범위에서 재택근무를 할 수 있다
 - ③ 중식은 구내식당에서 무료로 제공하며 ...
 - ④ 재택근무를 하려는 자는 희망일의 3근무일 전까지 신청
- ①은 "○일 전까지 신청" 구조가 똑같은 함정입니다
- ②는 "재택근무"가 들어있지만 답은 아닙니다

라운드 2 · "휴가 갈 때 회사에서 돈을 주나요?"

질문에 쓰인 단어가 문서에 하나도 없습니다.

"돈"으로 Ctrl+F 하면 → 0건

그런데 AI는 하계휴가 지원금(제12조)과
리프레시 휴가 지원금(제11조)을 골라냅니다.
→ 휴가 + 돈, 두 조건을 모두 본 것입니다

	1순위	2순위	3순위	4순위	내 순위와 달랐던 지점
내 답					
AI 답					

Ctrl+F로는 0건이었을 질문에서, 의미 검색은 정확한 조항을 찾아냅니다.
단어가 아니라 뜻으로 찾는다 — 이것이 임베딩이 하는 일의 전부입니다.

RAG 5단계 · 한 장 정리

▶ 오늘 배운 것을 이 그림 한 장으로 가져가세요



① ~ ③ 은 한 번만 해두는 준비 / ④ ~ ⑤ 는 질문할 때마다 반복

이 5개 이름만 알면 충분한 이유

10번 시간에 코드를 쓰는 사람은 Cursor입니다.
여러분은 5단계 중 어디를 바꿔달라고 말하면 됩니다.
예) "조각을 더 작게 잘라줘" / "10개 찾아줘"

스스로 점검 (각 항목 한 문장으로)

- ☐ AI가 우리 회사 일을 모르는 이유를 말할 수 있다
- ☐ 문서를 주면 답이 왜 달라지는지 말할 수 있다
- ☐ 청크 / 임베딩 / 벡터DB / 검색을 각각 한 문장으로 설명한다
- ☐ 다음 시간에 Cursor에게 시킬 문장을 써 두었다

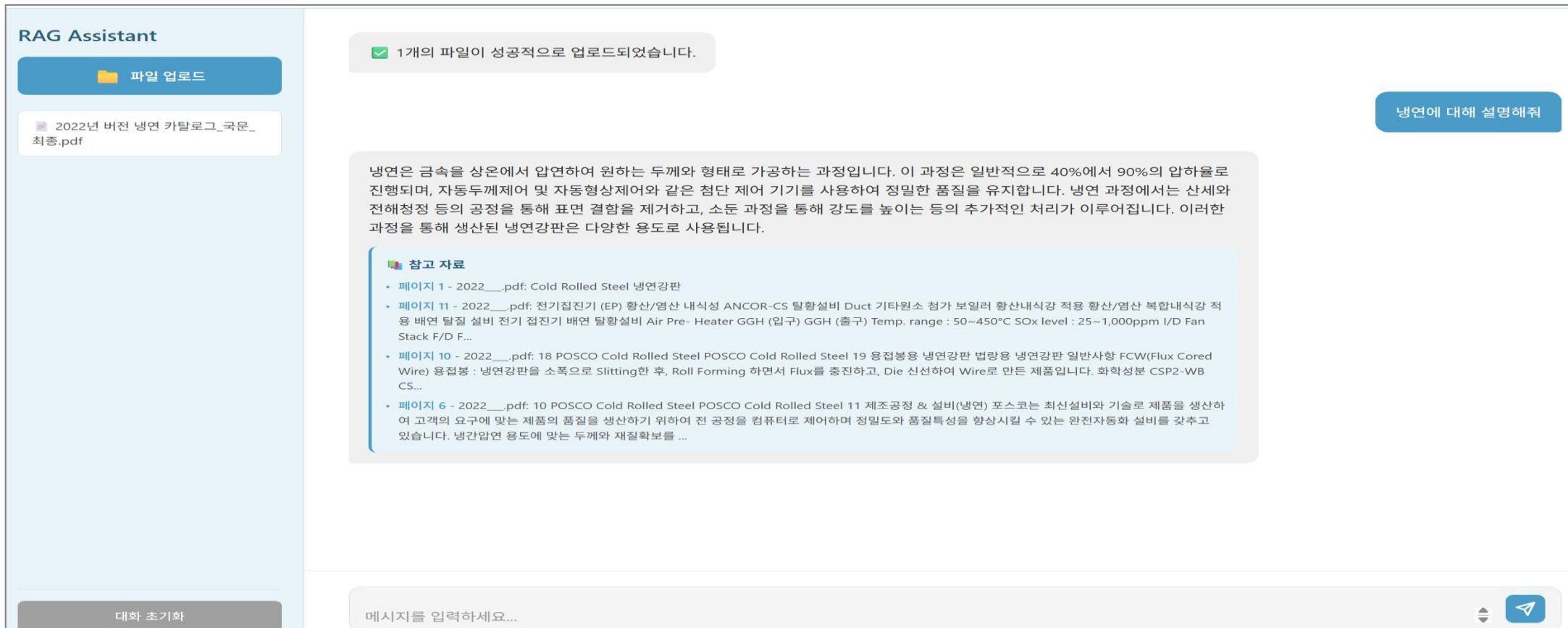
M3. 바이트코딩 활용 RAG 구현

8. RAG 구현

실습 1 : General Rag 구현

▶ 개요 : PDF 파일을 업로드하면 벡터 DB를 구성하고, 사용자의 질문에 대해 Chunk를 찾아서 답변 생성

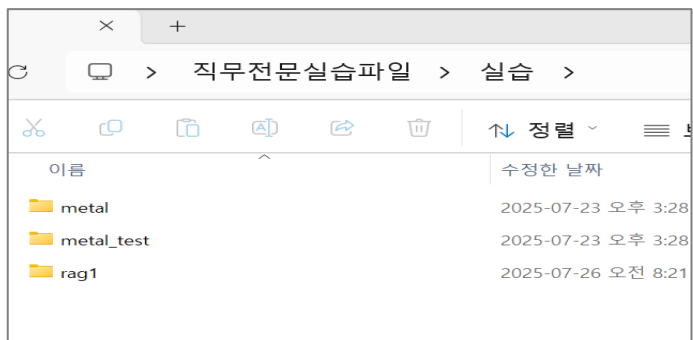
- GPT와 유사한 웹으로 구성, 텍스트는 500글자로 분할, 임베딩 모델 text-embedding-3-small , llm gpt-4o-mini
- chunk를 찾지 못하면 '모르겠습니다'라고 답변, 답변과 관련한 chunk 정보 출력, 대화 메모리 기능 사용



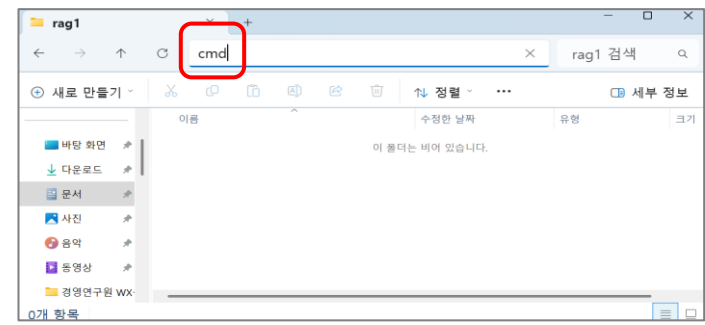
실습 1 : General Rag 구현

▶ 개발 환경 설정

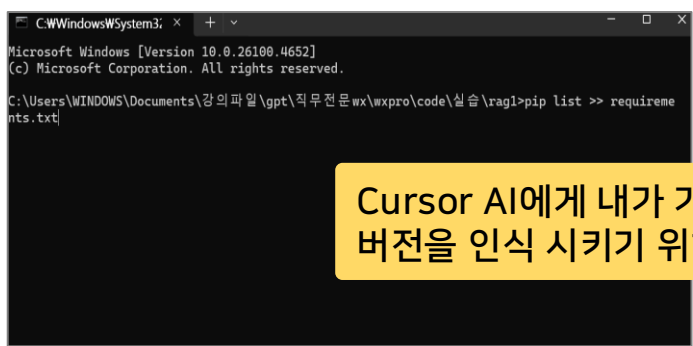
① 탐색기 : 실습 폴더 아래 'rag1' 폴더 생성



② 탐색기로 실습 폴더로 이동 → 주소창에 "cmd" 입력



③ "pip list >> requirements.txt" 입력



④ cursor AI : file → Open folder → "rag1" 폴더 클릭



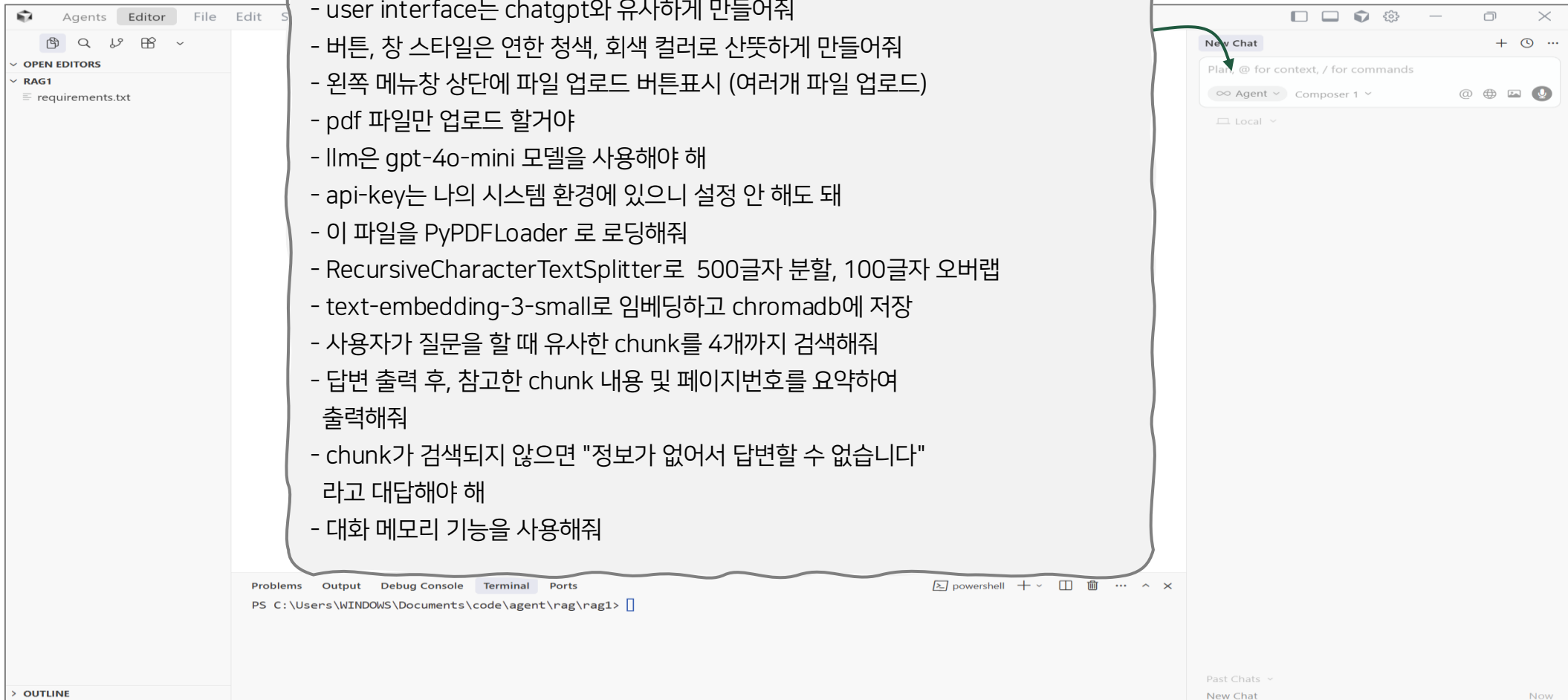
실습 1 : General Rag 구현

▶ 개발 환경 설정

⑤ -1 프롬프트 초안

Prompt

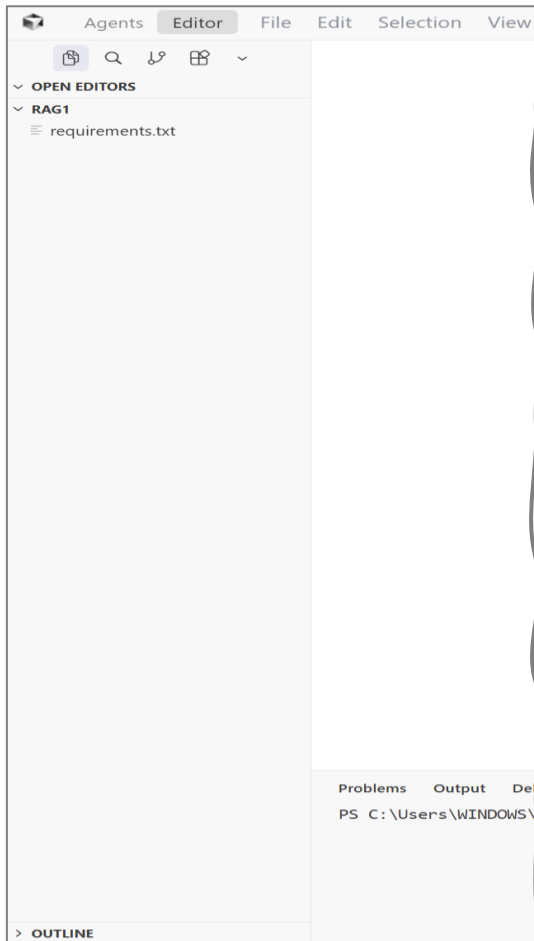
- 파이썬 langchain 패키지를 이용해서 rag를 만들어줘
- flask 패키지를 이용해 웹페이지를 만들어줘
- user interface는 chatgpt와 유사하게 만들어줘
- 버튼, 창 스타일은 연한 청색, 회색 컬러로 산뜻하게 만들어줘
- 왼쪽 메뉴창 상단에 파일 업로드 버튼표시 (여러개 파일 업로드)
- pdf 파일만 업로드 할거야
- llm은 gpt-4o-mini 모델을 사용해야 해
- api-key는 나의 시스템 환경에 있으니 설정 안 해도 돼
- 이 파일을 PyPDFLoader 로 로딩해줘
- RecursiveCharacterTextSplitter로 500글자 분할, 100글자 오버랩
- text-embedding-3-small로 임베딩하고 chromadb에 저장
- 사용자가 질문을 할 때 유사한 chunk를 4개까지 검색해줘
- 답변 출력 후, 참고한 chunk 내용 및 페이지번호를 요약하여 출력해줘
- chunk가 검색되지 않으면 "정보가 없어서 답변할 수 없습니다" 라고 대답해야 해
- 대화 메모리 기능을 사용해줘



실습 1 : General Rag 구현

▶ 개발 환경 설정

⑤ -2 대화창에 프롬프트 입력



Prompt(1/2)

RAG 기반 Flask 웹앱 개발 프롬프트

1. 목표

- 이 프로젝트는 PDF 문서를 업로드하고, 문서 내용을 기반으로 질의응답이 가능한 ****RAG 기반 웹 챗봇****을 만드는 것이다.
- 사용자 시나리오: ① 왼쪽 메뉴창에서 PDF 파일 업로드(다중 업로드 가능) → ② 채팅창에서 질문 입력 → ③ 답변 + 참고한 chunk 내용과 페이지 번호 확인
- UI/UX는 ChatGPT와 유사한 형태(왼쪽 사이드바 + 오른쪽 채팅 영역)로 구성한다.

2. 기술 스펙

환경 및 프레임워크

- Python 패키지: LangChain 사용
- 로컬 환경에 설치된 패키지 및 버전은 현재 폴더의 'requirements.txt'를 참고할 것
- 웹 프레임워크: Flask
- LLM: 'gpt-4o-mini' (API Key는 시스템 환경변수에 설정되어 있으므로 코드 내 별도 설정 불필요)

RAG 파이프라인

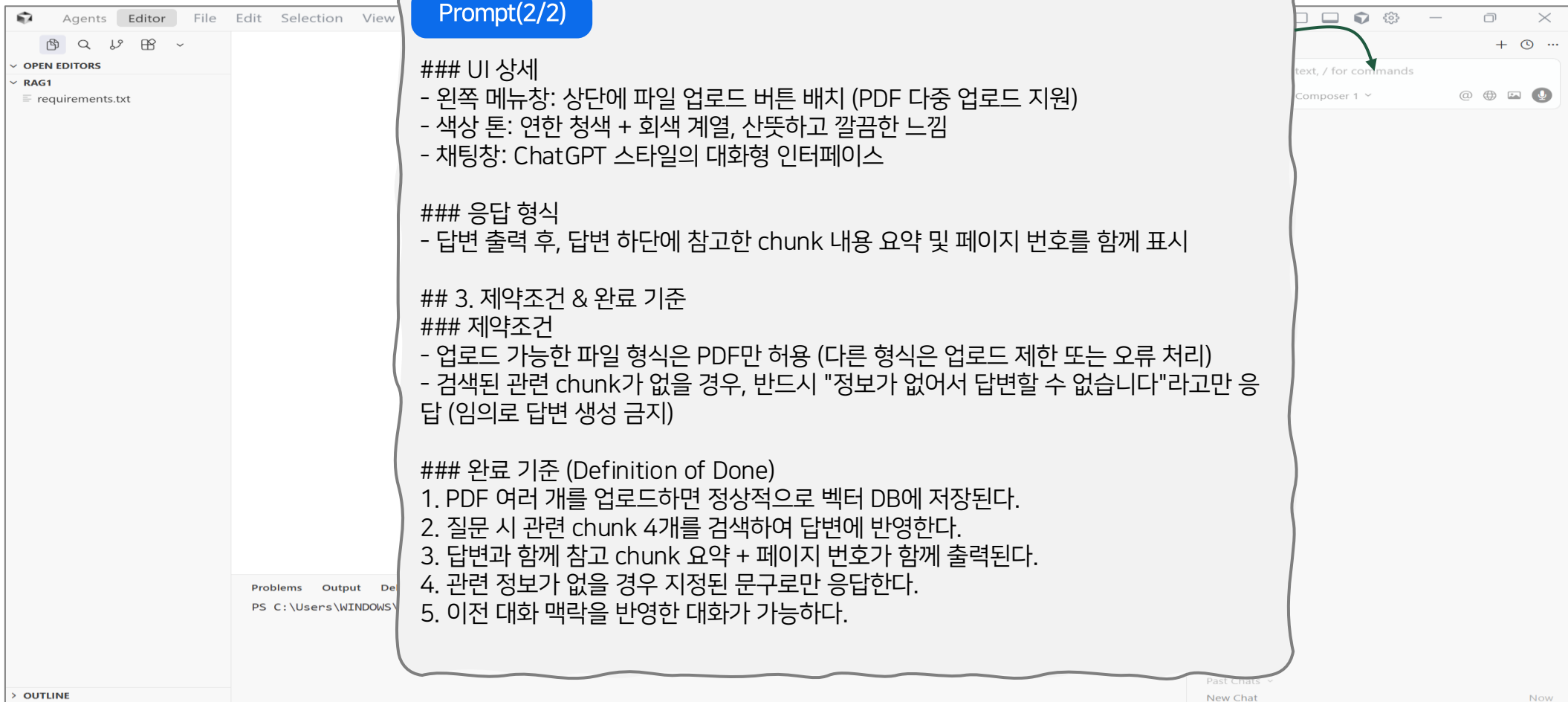
- 문서 로딩: 'PyPDFLoader'
- 청킹: 'RecursiveCharacterTextSplitter' (chunk size 500자, overlap 100자)
- 임베딩: 'text-embedding-3-small'
- 벡터 DB: ChromaDB
- 검색: 질문 시 유사도 기준 상위 4개 chunk 검색
- 대화 메모리 기능 포함 (이전 대화 맥락 유지)



실습 1 : General Rag 구현

▶ 개발 환경 설정

⑤ -2 대화창에 프롬프트 입력



The screenshot shows a VS Code editor window with a file named 'requirements.txt' open. A blue prompt box titled 'Prompt(2/2)' is overlaid on the editor, containing the following text:

UI 상세

- 왼쪽 메뉴창: 상단에 파일 업로드 버튼 배치 (PDF 다중 업로드 지원)
- 색상 톤: 연한 청색 + 회색 계열, 산뜻하고 깔끔한 느낌
- 채팅창: ChatGPT 스타일의 대화형 인터페이스

응답 형식

- 답변 출력 후, 답변 하단에 참고한 chunk 내용 요약 및 페이지 번호를 함께 표시

3. 제약조건 & 완료 기준

제약조건

- 업로드 가능한 파일 형식은 PDF만 허용 (다른 형식은 업로드 제한 또는 오류 처리)
- 검색된 관련 chunk가 없을 경우, 반드시 "정보가 없어서 답변할 수 없습니다"라고만 응답 (임의로 답변 생성 금지)

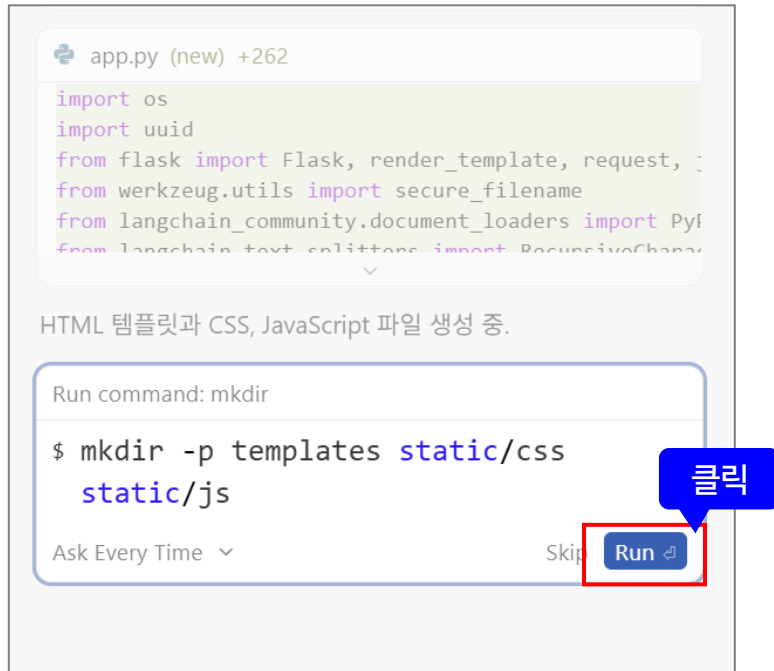
완료 기준 (Definition of Done)

1. PDF 여러 개를 업로드하면 정상적으로 벡터 DB에 저장된다.
2. 질문 시 관련 chunk 4개를 검색하여 답변에 반영한다.
3. 답변과 함께 참고 chunk 요약 + 페이지 번호가 함께 출력된다.
4. 관련 정보가 없을 경우 지정된 문구로만 응답한다.
5. 이전 대화 맥락을 반영한 대화가 가능하다.

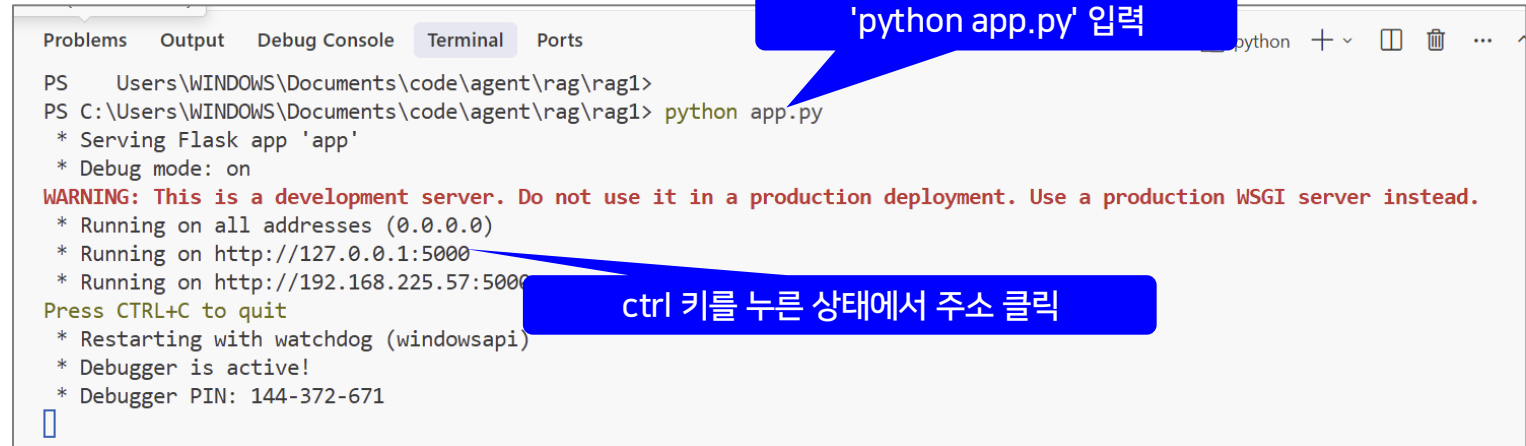
실습 1 : General Rag 구현

▶ 개발 환경 설정

⑥ 폴더생성, 패키지 설치 등 승인



⑦ 코딩이 완료되면 터미널 창에서 웹 실행



실습 1 : General Rag 구현

▶ 개발 환경 설정

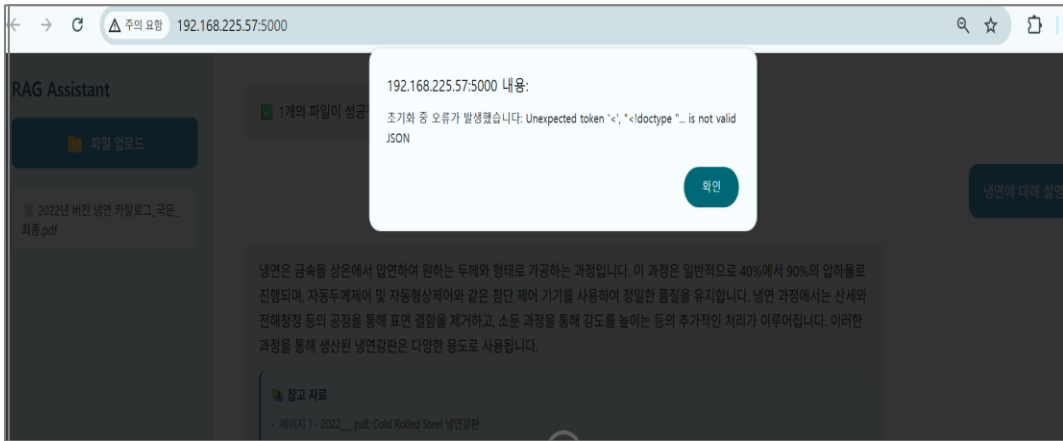
⑧ 웹 동작 상태 확인, 에러 발생시 수정

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:5000'. The page title is 'RAG Chat Assistant'. The main content area is titled 'RAG Assistant' and features a 'PDF 파일 업로드' (PDF File Upload) button. A blue callout bubble points to this button with the text: '2022년 버전 선재 카탈로그_국문_최종.pdf' 업로드. Below the upload button, the filename '2022년 버전 선재 카탈로그_국문_최종.pdf' is listed. To the right, there is a button labeled '피아노선재에 대해 알려줘' (Tell me about Piano Wire). The chat area displays a response from the assistant: '피아노선재는 고강도 Bead Wire, PC강선, 교량케이블 등에 주로 사용되는 고탄소 청정강입니다. 이 선재는 극세선 신선성, 고강도, 내피로성이 우수한 특성을 가지고 있습니다. 주요 규격으로는 POSMICRO62, POSCABLE82, POSCABLE86, POSCABLE90, POSCABLE92, POSCABLE98 등이 있으며, JIS 규격으로는 SWRS62A/B부터 SWRS92A 까지 다양합니다. 피아노선재는 주로 스프링강과 같은 고탄소강 제품으로, 높은 강도와 선 가공성을 필요로 하는 용도에 적합합니다.' Below the response, there is a section titled '참고 자료:' (Reference Materials) with three entries: '페이지 13: Micro Cable용 피아노 선재', '페이지 10: 16 POSCO Wire Rod', and '페이지 6: 8 POSCO Wire Rod'. The bottom of the interface shows a text input field with the placeholder '메시지를 입력하세요' (Enter message) and a '전송' (Send) button.

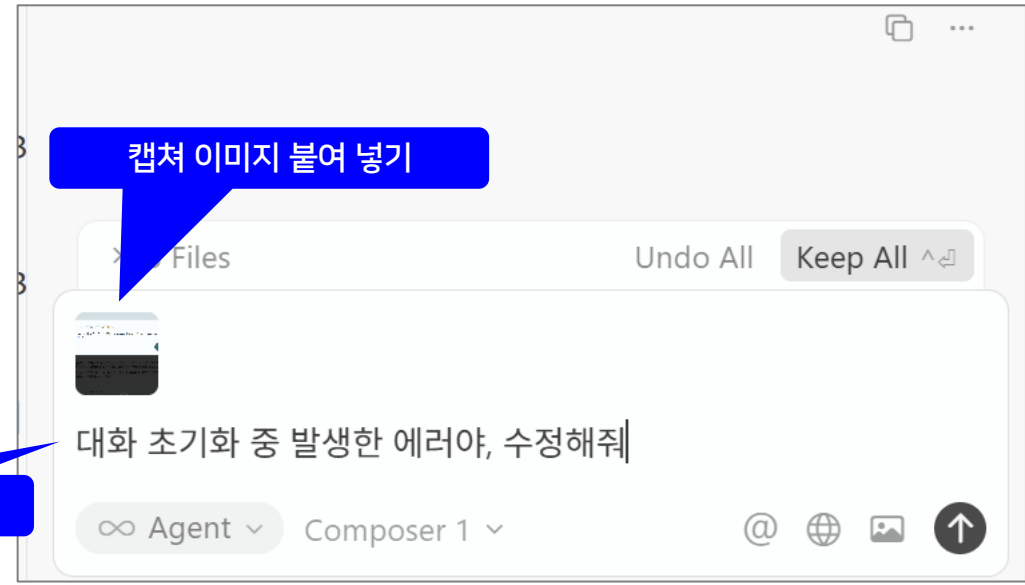
실습 1 : General Rag 구현

▶ 개발 환경 설정

⑨ 오류 수정: 오류가 발생한 화면을 캡처



→ cursor AI 대화창에 캡처 이미지 붙여 넣고 오류 상황 입력



수정 프롬프트

※ 코드 수정 완료 후 웹 페이지 새로고침하여 수정 여부 확인

실습 2 : Local LLM, 임베딩 모델 활용 Rag 구현하기

▶ Local LLM, 임베딩 모델



- 모델명 : [BAAI/bge-m3](#) (임베딩)
- 크기 : 1GB 이내
- 문서 벡터화, 검색, RAG Retriever
- 다국어, 한국어 지원 우수
- CPU/GPU 임베딩 사용
- 빠르고 정확한 검색 품질, 가벼움



- 모델명 : [EXAONE-3.5-2.4B-Instruct-GGUF](#) (LLM)
- 크기 : 1.8GB 이내
- 대화형 Instruct LLM, 오프라인 대화
- 한국어 강점, 한국어, 영어 이중언어 최적화
- GGUF 포맷, CPU 에서도 로컬 실행 가능
- 소형 모델이라 복잡한 추론에 한계

실습 2 : Local LLM, 임베딩 모델 활용 Rag 구현하기

▶ Local LLM, 임베딩 모델

종류	특징
EXAONE-3.5	LG AI Research의 EXAONE 3.5 모델 시리즈
2.4B	24억(2.4 billion) 개의 파라미터를 가진 모델
Instruct	지시문(Instruction)을 따르도록 추가 학습된 버전. 일반적인 채팅이나 질의응답에 적합
Q5_K_M	5비트 양자화(Quantization) 방식의 모델
.gguf	GGUF(GPT-Generated Unified Format) 파일 형식. llama.cpp, Ollama, LM Studio 등에서 사용하는 모델 포맷

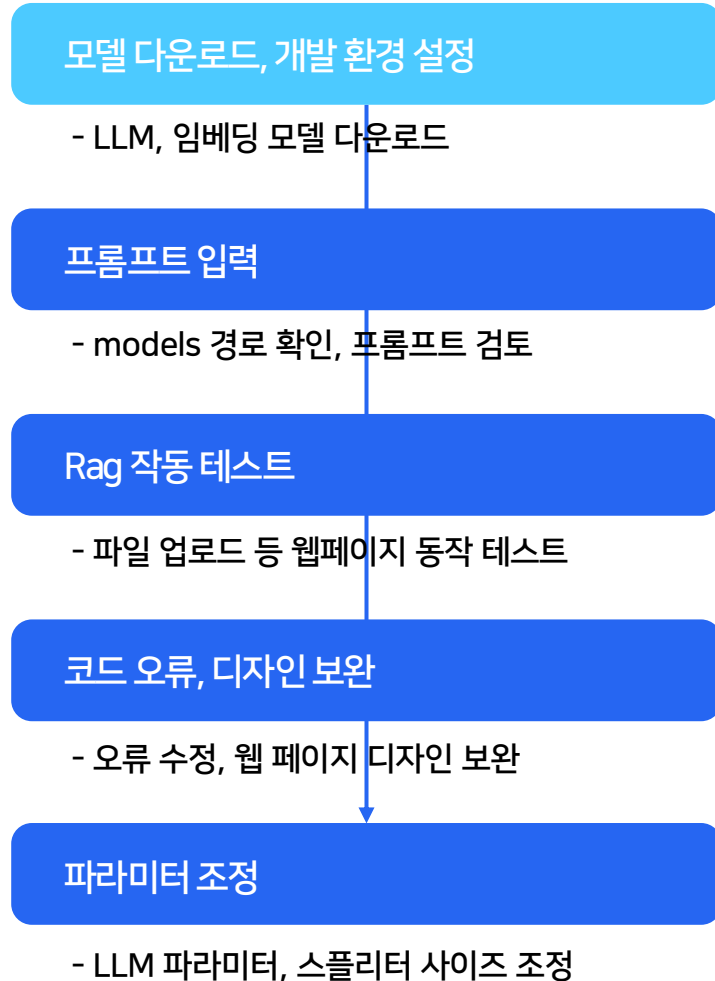
실습 2 : Local LLM, 임베딩 모델 활용 Rag 구현하기

▶ 모델 다운로드 및 개발 환경 설정

- ① 폴더 생성 : Rag 코드 생성할 폴더에 models 폴더 생성 ex) Rag 폴더 : `₩local_rag`, 모델 저장 폴더 : `₩local_rag₩models`
- ② 터미널 실행 : 탐색기로 `₩local_rag₩models` 폴더로 이동 후, 주소창에 'cmd' 입력
- ③ 허깅페이스 패키지 설치 : 터미널에 `'pip install huggingface_hub'` 입력
- ④ 임베딩 모델 다운로드 : 터미널에 명령어 입력
 - `₩local_rag₩models₩>위치에서 hf download BAAI/bge-m3 --local-dir bge-m3`
- ⑤ llm 다운로드 : 터미널에 명령어 입력
 - `₩local_rag₩models₩>위치에서 hf download LGAI-EXAONE/EXAONE-3.5-2.4B-Instruct-GGUF --include "*Q5_K_M.gguf" --local-dir exaone_2.4b`
- ⑥ requirements 파일 생성 : `₩local_rag` 폴더에서 명령어 입력
 - `₩local_rag₩>위치에서 pip list >> requirements.txt`
- ⑦ Cursor AI 실행 : file → Open folder → "`₩local_rag`" 폴더 클릭

실습 2 : Local LLM, 임베딩 모델 활용 Rag 구현하기

▶ Local rag 구현



Prompt 초안

파이썬 langchain 패키지를 이용해서 rag를 만들어줘

- flask 패키지를 이용해 웹페이지를 만들어줘
- user interface는 chatgpt와 유사하게 만들어줘
- 버튼, 창 스타일은 연한 청색, 회색 컬러로 산뜻하게 만들어줘
- 왼쪽 메뉴창 상단에 파일 업로드 버튼표시 (여러개 파일 업로드)
- pdf 파일만 업로드 할꺼야
- llama-cpp-python 사용해줘
- (llm 과 임베딩 모델 모두 local 에 저장되어 있음)
- llm은 `"/models/exaone_2.4b/EXAONE-3.5-2.4B-Instruct-Q5_K_M.gguf"` 이용해
- 임베딩 모델은 `"/models/bge-m3"` 을 이용해서 500글자 분할, 100글자 오버랩, chromadb에 저장
- 사용자가 질문을 할 때 유사한 chunk를 4개까지 검색해줘
- chunk가 검색되지 않으면 "정보가 없어서 답변할 수 없습니다" 라고 대답해야 해
- 대화 메모리 기능을 사용해줘
- 답변을 출력한 후, 참고한 chunk 정보도 같이 출력해줘

실습 2 : Local LLM, 임베딩 모델 활용 Rag 구현하기

Prompt 1/2

RAG 기반 Flask 웹앱 개발 프롬프트

1. 목표 & 컨텍스트

- 이 프로젝트는 PDF 문서를 업로드하고, 문서 내용을 기반으로 질의응답이 가능한 ****RAG 기반 웹 챗봇****을 만드는 것이다.
- 사용자 시나리오: ① 왼쪽 메뉴창에서 PDF 파일 업로드(다중 업로드 가능) → ② 채팅창에서 질문 입력 → ③ 답변 + 참고한 chunk 내용과 페이지 번호 확인
- UI/UX는 ChatGPT와 유사한 형태(왼쪽 사이드바 + 오른쪽 채팅 영역)로 구성한다.

2. 기술 스펙

환경 및 프레임워크

- Python 패키지: LangChain 사용
- 로컬 환경에 설치된 패키지 및 버전은 현재 폴더의 'requirements.txt' 파일에 있으며, 반드시 여기에 있는 패키지 및 버전만 사용할 것.
- 웹 프레임워크: Flask
- LLM 및 임베딩 실행 방식: 'llama-cpp-python' 사용 (LLM, 임베딩 모델 모두 로컬 파일로 보유)
- LLM: 로컬 GGUF 모델 사용 — 경로: './models/exaone_2.4b/EXAONE-3.5-2.4B-Instruct-Q5_K_M.gguf'

RAG 파이프라인

- 문서 로딩: 'PyPDFLoader'
- 청킹: 'RecursiveCharacterTextSplitter' (chunk size 500자, overlap 100자)
 - 분할 기준(separators)은 "문단 → 문장" 순으로 우선 적용
 - 예: 'separators=["\n\n", "\n", ".", "?", "!", ":", ";"]'
 - 문단 구분(빈 줄, 줄바꿈)을 최우선으로 분할하고, 문단 내에서는 문장 단위(마침표, 물음표, 느낌표 뒤)로 분할되도록 구성
- 임베딩: 로컬 임베딩 모델 사용 — 경로: './models/bge-m3' ('llama-cpp-python'으로 로딩)
- 벡터 DB: ChromaDB
- 검색: 질문 시 유사도 기준 상위 4개 chunk 검색
- 대화 메모리 기능 포함 (이전 대화 맥락 유지)

실습 2 : Local LLM, 임베딩 모델 활용 Rag 구현하기

Prompt 2/2

UI 상세

- 왼쪽 메뉴창: 상단에 파일 업로드 버튼 배치 (PDF 다중 업로드 지원)
- 색상 톤: 연한 청색 + 회색 계열, 산뜻하고 깔끔한 느낌
- 채팅창: ChatGPT 스타일의 대화형 인터페이스

응답 형식

- 답변 출력 후, 답변 하단에 참고한 chunk 내용 요약 및 페이지 번호를 함께 표시

3. 제약조건 & 완료 기준

제약조건

- 업로드 가능한 파일 형식은 PDF만 허용 (다른 형식은 업로드 제한 또는 오류 처리)
- 검색된 관련 chunk가 없을 경우, 반드시 "정보가 없어서 답변할 수 없습니다"라고만 응답 (임의로 답변 생성 금지)

완료 기준

1. PDF 여러 개를 업로드하면 정상적으로 벡터 DB에 저장된다.
2. 질문 시 관련 chunk 4개를 검색하여 답변에 반영한다.
3. 답변과 함께 참고 chunk 요약 + 페이지 번호가 함께 출력된다.
4. 관련 정보가 없을 경우 지정된 문구로만 응답한다.
5. 이전 대화 맥락을 반영한 대화가 가능하다.

실습 3 : 계약서 검토 Rag 구현

▶ 개요 : 계약서 약관을 Rag로 구성, 작성한 계약서를 업로드하면 문장을 하나씩 읽어 Rag와 비교 후 오류가 있으면 수정

- GPT와 유사한 웹으로 구성, 계약서 약관/ 계약서 PDF 업로드, 약관은 text-embedding-3-small로 임베딩, llm gpt-4o-mini 사용
- 계약서 문장을 한 줄씩 읽은 후 Chunk 검색 및 비교, 오류가 있다면 원문과 수정 문구 출력, 오류 없을 시 원문만 출력

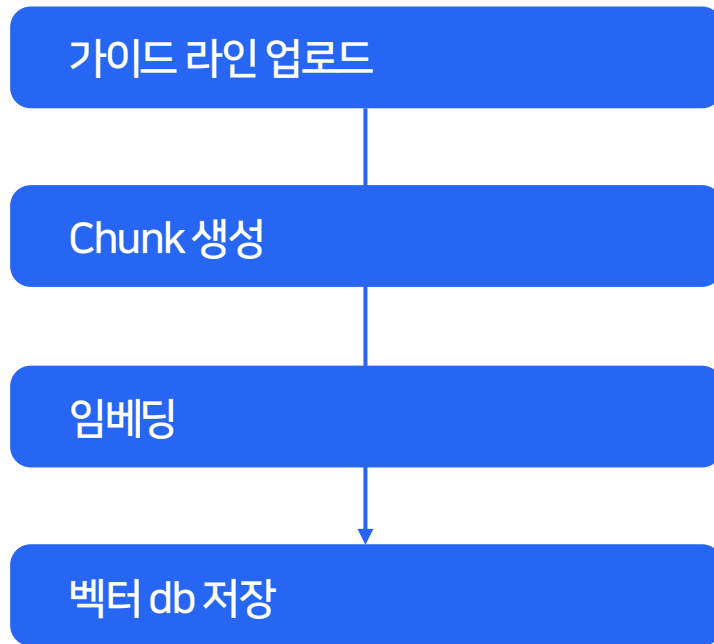
표준 공사 계약서	계약서 검토	메뉴
도급인 (주)포스코 (이하"갑"이라 한다)와 수급인 (주)한국철강(이하"을"이라한다)는 대등한 입장에서 서로 협력하여 신의에 따라 성실히 계약을 이행한다. 제1조 [공사내역] 1. 공 사 명 : 자재 창고 리모델링 2. 계약일자 : 2010년 1월 1일 3. 착공일자 : 2010년 1월 3일 4. 준공일자 : 2010년 1월10일 5. 공사기간 : 절대 공사기간을 (우천 및 천재지변 시를 제외한 나머지 공사기일) 준수한다. (공사 기간중 발생하는 변경, 협의지연, 확정지연 등 갑 측의 지연 사유로 인하여 연장된 일정은 제외한다.) 6. 공사내용 : 첨부 공사내역확정표 참조 제2조 [공사계약시 기성 지불계약] 1. 공사금액은 "부가가치세" 포함으로 하여 일금백만원정(₩1,000,000)으로 한다. 2. "갑"의 기성 지불내역은 아래사항과 같이 체결한다. 1) 계약금 (공사계약시) 10%(현금) 일금십만원정 (₩100,000) 2010년 1월 1일	2) 중도금 (공사착공후) 40%(현금) 일금사십만원정 (₩400,000) 2010년 1월 4일 3) 공사준공금 50%(현금) 일금오십만원정 (₩500,000) 2010년 1월10일 * 잔금 지급완료후 공사시설물을 갑에게 인수한다. * 계약금은 도면,견적서,현장실측,기타등등의 재변사항에 관한 비용임 3. 계약체결 및 작업완료 후라도 본 계약금액의 산출내역중 적용물량, 적용단가, 제비용 등의 적용이 과다 또는 과소하게 책정된 것이 발견된 때에는 도급인과 수급인은 합의하여 이를 감액 또는 증액 조정할 수 없다. 제3조 [계약금 합의사항] 1. 제출도면 및 내역서에 의거, 시공함을 원칙으로 하며 변경사항에 있어서는 "갑"과 "을"의 합의하여 수정할 수 있다.(단, 변경 및 추가공사 발생시에는 도면 및 금액을 별도 산출한다) (전체 공사에 관한 금액 기준은 1순위 내역서이며 2순위 도면을 기준으로 한다.) (내역서 및 도면에 포함되지 않은 공사는 인정하지 아니한다.) 2. 도급인이 설계변경을 요구한 경우에는 설계변경 당시를 기준으로 산정한 단가와 동단가에 낙찰율을 곱한 금액의 범위안에서 결정한다. 3. 도급인과 수급인의 공사착공 전 설계도면에 각자 서명하고, 갑은 수급인에게 제4조 [하	RAG 파일 업로드 계약서 검토 [계약서] 계약서 업로드 완료: pdf [원문] 표준 공사 계약서 도급인 (주)포스코 (이하"갑"이라 한다)와 수급인 (주)한국철강(이하"을"이라한다)는 대등한 입장에서 서로 협력하여 신의에 따라 성실히 계약을 이행한다. 제1조 [공사내역] 1. 공 사 명 : 자재 창고 리모델링 2. 계약일자 : 2010년 1월 1일 3. 착공일자 : 2010년 1월 3일 4. 준공일자 : 2010년 1월10일 [원문] 5. 공사기간 : 절대 공사기간을 (우천 및 천재지변 시를 제외한 나머지 공사기일) 준수한다. (공사 기간중 발생하는 변경, 협의지연, 확정지연 등 갑 측의 지연 사유로 인하여 연장된 일정은 제외한다.) 6. 공사내용 : 첨부 공사내역확정표 참조 제2조 [공사계약시 기성 지불계약] [원문] 1. 공사금액은 "부가가치세" 포함으로 하여 일금백만원정(₩1,000,000)으로 한다. 2. "갑"의 기성 지불내역은 아래사항과 같이 체결한다. 1) 계약금 (공사계약시) 10 %(현금) 일금십만원정 (₩100,000) 2010년 1월 1일 2) 중도금 (공사착공후) 40%(현금) 일금사십만원정 (₩400,000) 2010년 1월 4일 [원문] 3) 공사준공금 50%(현금) 일금오십만원정 (₩500,000) 2010년 1월10일 * 잔금 지급완료후 공사시설물을 갑에게 인수한다. * 계약금은 도면, 견적서, 현장실측, 기타등등의 재변사항에 관한 비용임 3. 계약체결 및 작업완료 후라도 본 계약금액의 산출내역중 적용물량, 적용단가, [원문] 제비용 등의 적용이 과다 또는 과소하게 책정된 것이 발견된 때에는 도급인과 수급인은 합의하여 이를 감액 또는 증액 조정할 수 없다. 제3조 [계약금 합의사항] 1. 제출도면 및 내역서에 의거, 시공함을 원칙으로 하며 변경사항에 있어서는 "갑"과 "을"의 합의하여 수정할 수 있다.(단, 변경 및 추가공사 발생 시에는 도면 및 금액을 별도 산출한다) [수정 문구] 계약서 문장이 가이더라인/약관에 위배됩니다. 따라서 수정이 필요합니다. 다음과 같이 수정할 수 있습니다: "제비용 등의 적용이 과다 또는 과소하게 책정된 것이 발견된 때에는 도급인과 수급인은 합의하여 이를 감액 또는 증액 조정할 수 있다. 제3조 [계약금 합의사항] 1. 제출도면 및 내역서에 의거, 시공함을 원칙으로 하여 변경사항에 있어서는 '갑'과 '을'의 합의하여 수정할 수 있다. (단, 변경 및 추가공사 발생 시에는 도면 및 금액을 별도 산출한다)" [원문] (전체 공사에 관한 금액 기준은 1순위 내역서이며 2순위 도면을 기준으로 한다.) (내역서 및 도면에 포함되지 않은 공사는 인정하지 아니한다.) 2. 도급인이 설계변경을 요구한 경우에는 설계변경 당시를 기준으로 산정한 단가와 동단가에 낙찰율을 곱한 금액의 범위안에서 결정한다. 3. 도급인과 수급인은 공사착공 전 설계도면에 각자 서명후 각 2부 보관한다. [원문] 제4조 [하자보수] 본 조항은 책임시공의 원칙하에 철저한 사후관리를 기초로한다. 1. 기본적으로 "갑"의 고의나 실수로 파손된 사항은 제외한다. 2. 하자보수 이행 기간은 준공일 기준 12개월로 한다. 3. 하자보수금은 계약금의 3%로 보증보험으로 대체할 수 있다. 제5조 [공사기간의 연장] [원문] 1. "갑"의 책임있는 사유 또는 천재지변, 불가항력으로 지연되는 경우 "을"은 공사기간을 연장할 수 있다. 2. 제1항의 규정에 의거 공사기간이 연장되는 경우 현장관리비등 추가경비는 설계변경 당시를 기준으로 산정한 단가로 한다. 제6조 [기성부분금] 1. 계약서 기성부분에 관하여 "을"은 기성 부분을 요청할 수 있으며, "갑"은 계약서 [원문] 에 명시한 바에 따라 지급하여야 한다. 2. "을"은 제 1항의 공사내역서(견적서)에 따라 기성금액을 요구할 수 있으며 "갑"은 계약서에 명시한 바에 따라 지급하여야 한다. 3. "갑"이 제 2항의 규정에 의한 기성금액의 지급을 지연하는 경우에는 "갑"은 사 유가 없는 한 계약의 목적물을 인도 받음과 동시에 공사대금을 지급하여야 한다. [원문] 제7조 [분쟁의 해결] 1. 계약에 규정된 것을 제외하고 발생한 문제는 계약당사자 쌍방의 합의에 의하여 해결한다. 2. 제1항의 규정에 의한 합의가 성립되지 못한경우 관례법률 규정에 의 하여 본 정조위원회의 조정에 의하여 분쟁을 해결한다. 제8조 [계약외의 사항] 1.이 계약에 정하지 아니한 사항은 신의, 성실에 따라 갑, 을이 합의하여 결정한다 [원문] 단 3. 수급인은 작업범위, 작업내용, 작업수령조건, 작업품질, 환경관리, 안전관리 및 기술적인 사항을 기술한 작업

계약서_오류정정실습	2025-07-24 오후 1:58	Adobe Acrobat 문서	134KB
시설공사계약약관	2025-07-18 오전 8:34	Adobe Acrobat 문서	891KB
협력계약일반약관	2025-07-18 오전 8:33	Adobe Acrobat 문서	605KB

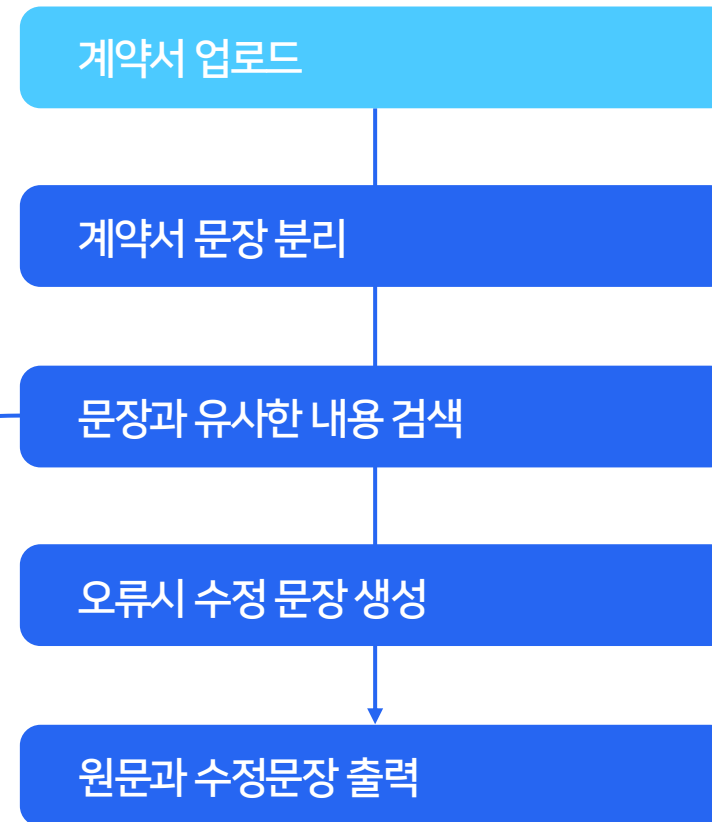
실습 3 : 계약서 검토 Rag 구현

▶ 흐름도

- 벡터db 흐름도



- 계약서 검토 흐름도



실습 3 : 계약서 검토 Rag 구현

▶ 프롬프트

Prompt

내가 만들려고 하는 웹 시스템은 아래와 같아

0. 제목 : LLM 기반 계약서 검토 웹

1. 개요

- 파이썬 langchain 0.3X 버전 패키지를 이용해서 LLM 기반 계약서 검토 웹 코드를 생성해줘
- 계약서 가이드라인이나, 약관을 업로드해서 rag를 구현하고, 사용자가 계약서를 업로드하면 한 문장씩 읽고, rag에서 유사 문서를 검색 후, 가이드라인이나 약관에 위배사항이 있는지 검토하고 계약서를 수정해서 출력하는 웹서비스를 구현하는 거야

2. 패키지 버전 및 환경

- flask 패키지를 이용해, llm은 gpt-4o-mini 모델을 사용해
- langchain 등 패키지 버전은 내 시스템에 설치된 'requirements.txt' 파일을 참조해줘
- api-key는 나의 시스템 환경에 있으니 설정 안 해도 돼
- 시스템 구현을 위해 추가적으로 필요한 패키지 버전은 알려줘
- 코드들은 이해하기 쉽게 주석을 추가해줘

3. 메뉴 구성

- user interface는 chatgpt와 유사하게 만들어줘 (왼쪽 : 메뉴, 오른쪽 : 채팅 및 결과)
- 화면 구성은 연한 파랑색과, 연한 회색을 이용해 줘
- 왼쪽 메뉴는 'rag 파일 업로드' 버튼 과 '계약서 업로드' 버튼이 있어야 해
- '계약서 업로드'가 완료되면 '계약서 검토' 버튼이 보여야 해
- 오른쪽 화면은 답변을 보여주고, 하단에는 일반적인 질문을 할 수 있는 질문창을 보이게 해줘
- rag 파일 업로드, 계약서 업로드시 답변 창에 진행율을 보여주고 완료 결과를 출력해줘 (tqdm 패키지 활용, 바 출력)

4. rag 구현

- 'rag 파일 업로드' 버튼을 누르면 pdf 파일을 업로드 할 수 있도록 구성해줘
- PyPDFLoader로 여러 개의 파일을 업로드 할 수 있어야 해
- 업로드한 문서를 RecursiveCharacterTextSplitter로 30글자로 분할, 5글자로 오버랩 해줘, separators는 ["\n", "\n\n"]
- text-embedding-3-small 로 임베딩하고 chromadb에 저장해

실습 3 : 계약서 검토 Rag 구현

▶ 프롬프트(계속)

5 계약서 검토 구현

- '계약서 업로드' 버튼을 누르면 PyPDFLoader로 pdf 파일을 업로드 할 수 있게 해줘
- 계약서를 RecursiveCharacterTextSplitter로 30글자로 분할, 오버랩 하지만, separators는 ["\n", "\n\n"]
- 계약서 문장과 rag 문서를 비교하여 계약서 문장에 오류 또는 보완이 필요한 부분을 찾은 후 수정해줘
- 계약서 문장을 '[원문]' 이라고 표시 후 출력하고 오류를 수정한 문장을 '[수정 문구]' 라고 표시한 후 진한 파란 글씨체로 원문의 바로 밑에 출력해줘 (7. 출력 예시 참조)
- 계약서 문장에 이상이 없다면 원문만 출력하면 됨
- 화면에 문장 하나씩 하나씩 출력해줘, 한꺼번에 출력되면 검토하는 동안 진행 여부를 확인할 수 없어

6. 기타

- 사용자가 일반적인 질문을 하게 되면 rag를 이용하지 말고 llm에 직접 요청해서 답변을 생성하면 되
- 만약 성능 향상을 위해 에이전트가 필요하다면 langgraph 패키지를 이용해서 에이전트를 구현해줘

7. 출력 예시

[원문] 3. 계약체결 및 작업완료 후라도 본 계약금액의 산출내역중 적용물량, 적용단가, 제비용 등의 적용이 과다 또는 과소하게 책정된 것이 발견된 때에는 도급인과 수급인은 협의하여 이를 감액 또는 증액 조정할 수 없다.

[수정문구] 3. 계약체결 및 작업완료 후라도 본 계약금액의 산출내역중 적용물량, 적용단가, 제비용 등의 적용이 과다 또는 과소하게 책정된 것이 발견된 때에는 도급인과 수급인은 협의하여 이를 감액 또는 증액 조정할 수 있다.

실습 3 : 계약서 검토 Rag 구현

▶ 실습절차

① 실습 폴더 생성

- 실습 폴더 아래 rag2 폴더 생성

② New Chat 클릭

- 사이드바에서 New Chat 클릭

Chunk 를 어떻게 나눌 것인가?
계약서 문장 분리를 어떻게 할 것인가? 에 따라 검토 품질이 달라짐

③ 프롬프트 입력

- 대화창에 프롬프트 입력, 진행 여부 등에 대해 답변

④ 웹 테스트

- 실습파일 업로드 후 웹 테스트, 이상 여부 확인
※ rag 문서 : 시설공사계약약관, 협력계약일반약관, 검토 계약서 : 계약서_오류정정실습)

⑤ 에러 보완

- 에러 수정 프롬프트 생성, 코드 생성 후 적용, 재 실행

M4. 종합 실습

9. 막혔을 때 빠져나오는 법

초보가 막히는 4가지 패턴

▶ 해결하지 못해서가 아니라, 전달하지 못해서 막힙니다



패턴 ① 화면만 본다

빨간 글씨를 눈으로만 보고
"에러 났어요"라고 말합니다.
AI는 그 화면을 못 봅니다.



패턴 ② "안 돼요"만 말한다

무엇을 하려다 어디서 안 됐는지
없이 결과만 전달합니다.
AI는 추측으로 헛수고합니다.



패턴 ③ 같은 말만 반복

"다시 해줘"를 10번 누릅니다.
코드는 점점 복잡해지고
원래 되던 것까지 망가집니다.



패턴 ④ 돌아갈 곳이 없다

잘 되던 시점의 복사본이 없어
처음부터 다시 만듭니다.
2시간이 날아갑니다.

바이브코딩에서 잘하는 사람과 못하는 사람의 차이는 코딩 지식이 아니라
① 상황을 정확히 전달하는 능력 ② 언제든지 되돌아갈 수 있는 습관 — 이 둘입니다.

오늘의 약속

막히면 혼자 20분 헤매지 말고 — ① 멈추고 ② 복사하고 ③ 설명한다. 이 3단계를 몸에 붙이는 것이 이 시간의 전부입니다.

에러 대응 3단계 — STOP · COPY · TELL

▶ 막힌 순간 30초 안에 할 일

STOP 멈춘다

- 더 이상 새 요청을 넣지 않는다
- 화가 난 상태로 "다시!"를 누르지 않는다
- 지금 화면을 그대로 둔다

COPY 통째로 복사한다

- 터미널의 빨간 글씨 전문(全文)
- 스크린샷 말고 반드시 텍스트로
- 마지막으로 실행한 명령어도 함께

TELL 상황을 설명한다

- 아래 5요소 템플릿을 채운다
- 추측("아마 DB 문제인 듯")은 쓰지 않는다
- 사실만 적는다

상황 설명 5요소 템플릿

- ① 목표 — 무엇을 만들려고 했나
- ② 방금 한 일 — 어떤 요청/명령을 넣었나
- ③ 기대 — 무엇이 나오길 바랐나
- ④ 실제 — 무엇이 나왔나 (화면 문구 그대로)
- ⑤ 에러 전문 — 빨간 글씨 전체 붙여넣기

※ ⑤만 있어도 절반은 해결됩니다. ①~④는 해결 속도를 높입니다.

이렇게 말하면 AI도 헤맷니다

"안 돼요. 고쳐주세요"

"아까는 됐는데 지금은 이상해요"

이렇게 말하면 한 번에 잡힙니다

"PDF 번역 웹페이지를 만들고 있어. 방금 '파일 업로드 기능 추가해줘'라고 요청하고 실행했어. 업로드 버튼이 보이길 기대했는데 화면이 하얗게 비어. 터미널 에러 전문은 아래와 같아: (붙여넣기)"

【실습】고장난 앱 고치기

- ▶ 강사가 배포한 프로젝트 폴더에는 버그 3개가 심어져 있습니다. 코드는 직접 고치지 않습니다
— 상황을 전달하고 Cursor에게 시키세요.

버그 ①

실행하자마자 멈춤

증상 : ModuleNotFoundError
No module named ...

할 일 : 에러 전문을 그대로 붙이고
"무엇이 문제이고 어떻게 고치는지"
물어본 뒤 Cursor에게 시키기

버그 ②

화면은 뜨는데 답이 안 옴

증상 : 401 Unauthorized 또는
Incorrect API key

할 일 : 401이 무슨 뜻인지 묻고,
노트북 환경변수의 키를 쓰도록
Cursor에게 고치게 하기

버그 ③

수정했는데 화면이 그대로

증상 : 변경이 반영되지 않음
(에러 메시지가 전혀 없음)

할 일 : "방금 무엇을 바꿨고 무엇이
그대로인지" 5요소로 설명하고
확인 방법 물어보기

진행 방법

- ① 개인 10분 — 혼자 힘으로 3개 모두 시도
- ② 짝 5분 — 못 잡은 것을 옆 사람에게 설명하고 협업 해결
- ③ 전체 5분 — 팀별로 "어떻게 물었더니 풀렸는지" 공유

실습 중 금지사항

- 소스 파일을 열어 직접 고치기 (오늘은 Cursor를 시키는 연습)
- API 키·사번·개인정보를 채팅창에 붙여넣기
- 같은 요청 3회 이상 반복 (다음 장 참고)

목표는 "3개를 다 고치는 것"이 아니라 "막혔을 때 물어보는 법이 몸에 붙는 것"입니다.

같은 자리에서 3번이면 전략을 바꿔라

▶ 반복 요청은 문제를 푸는 게 아니라 코드를 망가뜨립니다

- 1

새 채팅으로 옮긴다
 대화가 길어지면 AI가 앞의 실패한 시도들에 끌려다닙니다.
 새 채팅에 목표와 현재 코드만 다시 주세요.
- 2

문제를 작게 만든다
 전체 앱 말고 안 되는 기능 하나만 떼어 최소 예제로.
 "이 부분만 따로 동작하는 코드를 만들어줘"
- 3

되돌린다
 마지막으로 정상 동작하던 시점으로 복귀.
 Cursor의 되돌리기 또는 복사해 둔 폴더 사용.
- 4

다른 방식을 요청한다
 "같은 방법으로 고치지 말고, 아예 다른 방식으로 다시 설계해줘"라고 명시적으로 요청.

❌ 하지 말아야 할 것

"다시 해줘 / 아직도 안 돼 / 제대로 좀 해줘"를 반복하면 AI는 기존 코드 위에 땀질을 쌓습니다.

🕒 3분 룰

같은 에러를 3번 봤거나 3분이 지났으면, 위 4수 중 하나로 반드시 전환합니다.

되돌릴 수 있게 만들어 두세요 — 잘 동작하는 순간마다 폴더를 통째로 복사(v1_작동, v2_업로드추가).

5분 전으로 돌아갈 수 있으면, 어떤 실수도 5분짜리 실수가 됩니다.

먼저 쓰면 덜 막힌다 — 요구사항 5줄

▶ 막힘의 절반은 "무엇을 만들지"가 흐릿해서 생깁니다

요구사항 5줄 템플릿

- ① 누가 쓰나 — 사용자는 누구인가
- ② 무엇을 하나 — 한 문장으로
- ③ 입력 — 무엇을 넣으면
- ④ 출력 — 무엇이 나오나
- ⑤ 안 할 것 — 이번엔 만들지 않는 기능

⑤가 가장 중요합니다. 범위를 막지 않으면 AI가 기능을 계속 늘려 감당 못 할 코드를 만듭니다.

작성 예시 — 회의록 정리 도구

- ① 우리 팀 회의 주관자
- ② 회의록 텍스트에서 할 일만 뽑아 표로 정리한다
- ③ 회의록 txt 파일 1개
- ④ 할 일 / 담당자 / 기한 3열 표, todo.md로 저장
- ⑤ 이번엔 메일 발송·캘린더 연동은 만들지 않는다

→ 이 5줄을 Cursor 첫 메시지로 그대로 붙여넣습니다.

막혔을 때 순서 체크리스트 (인쇄해서 모니터 옆에 두기)

- ① 멈춘다 — 새 요청을 넣지 않는다 ② 에러 전문을 텍스트로 복사한다
- ③ 5요소로 설명한다(목표·한 일·기대·실제·에러) ④ 답을 받으면 한 번에 하나씩만 적용한다
- ⑤ 되면 즉시 폴더 복사본을 만든다 ⑥ 3번/3분 넘으면 새 채팅·문제 축소·되돌리기·다른 방식
- ⑦ 그래도 안 되면 옆 사람에게 5요소로 설명한다 (설명하다가 스스로 찾는 경우가 많습니다)

M4. 종합실습

10. AI Agent 맛보기

챗봇과 에이전트는 무엇이 다른가

▶ 레시피를 읽어주는 AI vs 주방에 들어가 요리하는 AI

챗봇 (Chatbot)

"김치찌개 만드는 법 알려줘"

→ 레시피를 알려준다. 끝.

요리는 여전히 내가 한다. 재료도 내가 꺼내고,
불도 내가 켜고, 맛도 내가 본다.

에이전트 (Agent)

"김치찌개 끓여줘"

→ 냉장고를 열고, 재료를 꺼내고, 불을 켜고,
중간에 맛을 보고, 싱거우면 소금을 더 넣고,
다 되면 "완성됐습니다"라고 보고한다.

	챗봇	에이전트
하는 일	묻는 말에 답한다	목표를 받아 스스로 단계를 밟는다
결과물	말(텍스트)	실제로 바뀐 파일·시스템·결과물
도구 사용	없음	파일 읽기/쓰기, 실행, 검색 등
사람 개입	매 턴마다 필요	중요한 순간에만 확인·승인

한 줄 정의 — 에이전트는 "목표를 주면, 될 때까지 도구를 써가며 스스로 반복하는 AI"입니다.
핵심 차이는 똑똑함이 아니라 「다음에 무엇을 할지 스스로 정하는가」입니다.

에이전트의 4단계

▶ 한 바퀴가 아니라, 될 때까지 돕니다



① 계획

지금 무엇을 먼저 할지 정한다
결과를 보고 다음 수를 바꾼다



② 도구 사용

필요한 도구를 골라 쓴다
안 써도 되면 건너뛴다



③ 실행

실제로 파일을 읽고 쓰고
명령을 돌려 결과를 받는다



④ 검수

결과가 기준에 맞는지 스스로 본다
아니면 ①로 돌아간다

 여러분은 이미 에이전트를 썼습니다 — Cursor에게 "로그인 기능 추가해줘"라고 했을 때

①계획 무엇부터 할지 정하고 → ②도구 기존 파일을 열어 읽고 → ③실행 코드를 고쳐 쓰고 돌려보고
→ ④검수 에러가 났는지 확인하고 → ①계획 방금 실패한 방법을 기억한 채 다시 정하고 → 완료 보고
챗봇이었다면 ①에서 코드만 알려주고 끝났을 것입니다. 나머지는 전부 여러분 몫이었겠죠.

 4단계가 보인다고 다 에이전트는 아닙니다

화면에 [계획] [도구] [실행] [검수]를 찍는 것은 고정된 코드로도 됩니다. 갈리는 지점은 하나입니다.

다음에 무엇을 할지를 「파이썬 코드」가 정하면 자동화 스크립트, 「AI」가 매 단계 정하면 에이전트입니다.

도구(Tool)와 MCP

▶ AI에게 「손」을 달아주는 표준 규격

MCP란 무엇인가

AI가 외부 도구·데이터에 접근하는 방식을 통일한 규격입니다.

- 비유 — USB-C. 기기마다 다른 충전기를 쓰던 것을 하나로 통일
- 덕분에 새 도구를 붙일 때마다 새로 개발할 필요가 없음
- Cursor·Claude 등에서 설정만으로 도구를 추가할 수 있음

붙일 수 있는 도구의 예

- 내 PC의 파일과 폴더 (읽기·쓰기·정리)
- 사내 문서 저장소 · 위키 · 메일
- 데이터베이스 조회
- 웹 검색 · 사내 시스템 API
- 그리고 7번에서 배운 RAG 검색기도 「도구」의 하나입니다

도구를 붙이기 전에 반드시 정할 것 — 권한과 범위

에이전트에게 「손」을 준다는 것은 실제로 파일을 바꾸고 명령을 실행할 수 있게 한다는 뜻입니다.

- 읽기만 허용할지, 쓰기까지 허용할지 · 어느 폴더까지 접근하게 할지 · 삭제·전송은 사람 승인 후에만
- 사내 데이터를 외부로 보내는 도구는 보안 검토를 거친 것만 사용

오늘 실습에서는

별도 MCP 설정 없이, Cursor가 기본으로 가진 「내 프로젝트 폴더의 파일을 읽고 쓰는 도구」만 사용합니다. 그것만으로도 충분한 에이전트가 됩니다.

좋은 요구사항서의 4요소

▶ 에이전트에게는 "무엇을"보다 "어디까지"가 더 중요합니다

1. 역할 — 너는 무엇을 하는 담당인가

"너는 회의록에서 실행 과제를 뽑는 담당이야"

2. 도구·범위 — 무엇을 쓸 수 있는가

"도구 4개를 만들고, 에이전트가 골라 쓰게 해"

3. 규칙 — 지킬 것과 하지 말 것

"없는 담당자·날짜를 지어내지 마. 모르면 미정"

4. 종료조건 — 언제 끝난 것으로 보는가

"검수 통과하면 끝. 최대 8단계, 재시도 2회"

❌ 나쁜 요구사항

"회의록 좀 정리해줘"

- 무엇을 정리할지, 어디에 쓸지, 언제 끝인지 알 수 없습니다.
- 엉뚱한 파일을 건드리거나 끝없이 되묻습니다.

✅ 좋은 요구사항

4요소가 모두 들어간 지시서

- 한 번에 끝나고 결과가 예측 가능합니다.
- 무엇을 확인해야 하는지도 분명합니다.

그리고 에이전트에게만 필요한 다섯 번째 항목 —

「다음에 무엇을 할지는 코드가 아니라 네가 정해라」. 이 한 줄이 없으면 에이전트가 안 됩니다.

【실습】내 첫 업무 에이전트 만들기

▶ 계획 → 도구사용 → 실행 → 검수가 스스로 도는 프로그램을 만듭니다

배포 폴더에 코드는 없습니다. 회의록 · 임직원 명부 · 요구사항서뿐 — 프로그램은 여러분이 만듭니다.

만들 때 — 도구 4개를 가진 에이전트

- ① 회의록_읽기() 고른 회의록을 읽는다
 - ② 담당자_조회(표기) 명부에서 정식 이름·부서를 찾는다
 - ③ 과제_저장(목록) todo.md로 저장한다
 - ④ 결과_검수() 저장한 결과를 다시 읽어 5가지를 점검
- 4개를 순서대로 부르는 게 아니라, 매 단계 AI가 골라 쓰게 만듭니다

회의록에 심어둔 장치 — AI가 판단해야만 풀립니다

- "이 책임" → 명부에 2명(이수진·이강우) → 미정 + 사유
- 교육장 예약 · 수료 기준 → 담당자 없음 → 미정
- "다음 주까지" → 날짜로 바꾸지 말 것

실행할 때 — 브라우저에 뜨는 것

-  계획 다음에 무엇을 할지 AI가 말한다
 -  도구 4개 중 무엇을 골랐는지 + 왜
 -  실행 도구가 돌아간 결과
 -  검수  통과(초록) /  실패(빨강)
- 빨간  바로 다음에  가 나오면 성공입니다

검수에서 볼 것

- 담당자 미정을 지어내지 않았는가
- 비고 사유가 엉뚱한 항목에 복사되지 않았는가
- 원본 2개가 그대로인가 / 키가 코드에 없는가

오늘 만든 것은 "AI를 한 번 부르는 프로그램"이 아닙니다.

스스로 계획하고, 도구를 고르고, 자기 결과를 검수해 다시 도는 프로그램입니다.

내 업무에 붙일 곳 찾기

▶ 오늘 배운 내용을 조합하면 대부분의 사무 업무에 닿습니다



회의록 → 실행과제

녹취/메모에서 할 일·담당자·기한 추출



보고서 초안 자동화

정해진 양식에 데이터만 갈아 끼우기



자료 폴더 정리

규칙에 따라 파일명 변경·분류·중복 정리



사내 규정 Q&A 봇

RAG로 규정집을 근거로 답하게 하기



데이터 취합

흩어진 엑셀·CSV를 하나로 모아 요약



메일·공지 초안

상황만 주면 톤에 맞춰 초안 생성



점검 리스트 자동화

체크 항목 생성 → 결과 기록 → 요약



반복 문서 양식화

매주 같은 문서를 템플릿+데이터로 자동 생성



지금 1분 — 내 업무에서 하나만 고르기

내가 매주 반복하는 일 중, 규칙이 분명하고 결과물이 파일인 것: _____

그것을 에이전트에게 맡긴다면 지시서 4요소는? 역할 _____ 범위 _____ 규칙 _____ 종료조건 _____

「반복적이고 · 규칙이 분명하고 · 결과가 파일인」 업무가 에이전트로 가장 좋습니다.